إعداد عربي تعليمي · الدفعة الثانية

كيف يعمل لينكس

من الصورة العامة إلى سطر الأوامر

استنادًا إلى كتاب

برايان وارد

How Linux Works
What Every Superuser Should Know
Second edition · No Starch Press

> **مقدمة · الفصلان الأول والثاني · معجم · مختبرات**
> مسودة تعليمية جزئية، وليست ترجمة حرفية أو رسمية.
> الفصول من الثالث إلى السابع عشر لم تُعد بعد.

7 سبتمبر 2026 · Markdown + PDF

محتويات هذه الدفعة

مسار تأسيسي بالعربية: اقرأ الفكرة، شاهد المخطط، ثم جرّب ولاحظ.

مقدمة الطبعة العربية التعليمية 4

الفصل الأول: الصورة العامة 9

قبل أن نبدأ 9

1.1 مستويات التجريد وطبقاته في نظام لينكس 10

1.2 العتاد: فهم الذاكرة الرئيسية 11

1.3 النواة 13

1.4 فضاء المستخدم 18

1.5 المستخدمون 19

1.6 إلى أين نتجه بعد ذلك؟ 21

مختبر عملي: شاهد الصورة على جهازك 21

اختبر فهمك 24

خلاصة الفصل 25

الفصل الثاني: الأوامر الأساسية وبنية الأدلة 26

من الصورة العامة إلى التجربة 26

2.1 صدفة بورن: المسار /bin/sh 27

2.2 استخدام الصدفة 28

2.3 أوامر أساسية 30

2.4 التنقل بين الأدلة 32

2.5 أوامر متوسطة المستوى 35

2.6 تغيير كلمة المرور وصدفة الدخول 39

2.7 الملفات النقطية 39

2.8 متغيرات الصدفة ومتغيرات البيئة	40
2.9 مسار البحث عن الأوامر	41
2.10 المحارف الخاصة	42
2.11 تحرير سطر الأوامر	43
2.12 محررات النصوص	44
2.13 الحصول على المساعدة والتوثيق	44
2.14 إدخال الصدفة وإخراجها	46
2.15 فهم رسائل الخطأ	48
2.16 عرض العمليات والتعامل معها	50
2.17 أنماط الملفات وأذونات الوصول	53
2.18 أرشفة الملفات وضغطها	57
2.19 أساسيات بنية الأدلة في لينكس	61
2.20 تشغيل أوامر بصلاحيات المستخدم الأعلى	64
2.21 إلى أين نتجه بعد ذلك؟	66
مختبر ختامي: خمس تجارب مترابطة	66
أسئلة للمراجعة	69
خلاصة عملية قبل إغلاق الطرفية	71
معجم الدفعة الثانية	72
ملاحظات الإعداد والمراجع	79
خريطة إنجاز الكتاب	83

العناوين روابط داخلية. الأرقام تشير إلى صفحات هذه النسخة العربية، بما فيها الغلاف. إحالات المصدر في المتن تشير إلى صفحات الملف الإنجليزي. يمكنك كذلك التنقل من العلامات المرجعية في قارئ PDF.

مقدمة الطبعة العربية التعليمية

> **الدفعة الثانية — مسودة تراكمية للمراجعة.** تتضمن إعادة صياغة تعليمية لمقدمة الكتاب وفصليه الأول والثاني، مع معجم ومختبرات وإجابات. الفصول من الثالث إلى السابع عشر **لم تُعد بعد.** هذه ليست ترجمة حرفية، ولا نسخة عربية مكتملة. أُضيف الفصل الثاني بإذن المستخدم، وستُراجع المقدمة والفصلان معًا؛ لا تعني إتاحة هذه الدفعة اعتمادًا نهائيًا للنص السابق.

عن هذا الإعداد

يحمل الكتاب الأصلي عنوان How Linux Works: What Every Superuser Should Know، وهو من تأليف **برايان وارد (Brian Ward)** ونشر **No Starch Press**. يمكن صياغة عنوانه عربيًا هكذا: **كيف يعمل لينكس: ما ينبغي لكل مستخدم ذي صلاحيات عليا أن يعرفه.** يستند هذا المشروع إلى ملف الطبعة الثانية الموجود في المستودع، لا إلى طبعة أحدث.

الهدف هنا أن يصبح فهم لينكس ممكنًا لقارئ عربي يعرف استخدام الحاسوب، لكنه لم يدرس أنظمة التشغيل دراسة متخصصة. نحافظ على تسلسل الأفكار والمفاهيم وأرقام أقسام الفصل، ونشرحها بعربية فصحى واضحة. نضيف أمثلة صغيرة وأسئلة تساعد على الفهم، بدل أن نستبدل كل كلمة إنجليزية بكلمة عربية ونترك المعنى غامضًا.

النص التالي **إعداد تعليمي مستند إلى الكتاب**، وليس كلامًا جديدًا منسوبًا إلى مؤلفه. الفقرات الموسومة «إضافة تعليمية» و«توضيح حديث»، وكذلك المختبر والأسئلة والمعجم، من الإعداد العربي. والرسوم مخططات تعليمية جديدة مستندة إلى أفكار رسوم الفصل، لا نسخ مطابقة لها. تُذكر مصادر التوضيحات التقنية في ملاحظات الإعداد والمراجع.

تعود حقوق النص الأصلي إلى أصحابه. لا يتضمن هذا المشروع ادعاء بأنه ترجمة رسمية أو معتمدة، ولا يمنح بذاته إذنًا بترجمة الكتاب أو إعادة توزيعه؛ ينبغي التحقق من حقوق النشر والتوزيع قبل نشر هذه المادة خارج نطاق العمل عليها.

لماذا نتعلم ما وراء الواجهة؟

الفكرة التي تنطلق منها مقدمة الكتاب بسيطة: من حقك أن تفهم ما يفعله حاسوبك، وأن تستخدم برامجه لتحقيق ما تريد، ضمن حدود قدراتها. ليس المطلوب أن تحفظ مئات الأوامر، بل أن تملك تصورًا يفسر ما يحدث عندما يعمل الجهاز بصورة طبيعية، وما قد يكون حدث عندما يتعطل شيء فيه.

يصلح لينكس لهذا النوع من التعلم لأنه يتيح لك فحص كثير من مكوناته مباشرة. يمكنك قراءة ملفات إعداد نصية، ومشاهدة العمليات الجارية، وتتبع استخدام الموارد، والرجوع إلى توثيق الأدوات. لا يعني ذلك أن كل شيء بسيط، بل إن النظام لا يشترط عليك الاكتفاء بواجهة تخفي تفاصيله.

المشكلة الأولى ليست قلة المعلومات، بل كثرتها: أي ملف يضبط هذه الخدمة؟ من بدأ هذه العملية؟ هل المشكلة في البرنامج أم في الشبكة أم في جهاز التخزين؟ يساعدك فهم العلاقة بين الأجزاء على طرح السؤال الصحيح قبل البحث عن الأمر المناسب.

> **إضافة تعليمية:** عندما تنسخ أمرًا من الإنترنت، اسأل ثلاثة أسئلة: ماذا يقرأ؟ ماذا يغيّر؟ وبصلاحيات مَن يعمل؟ إذا لم تعرف الإجابة، فافهم الأمر أولًا، ولا تجعل صلاحيات المدير وسيلة لتجاوز كل رسالة خطأ.

لمن كُتبت هذه المادة؟

يفيد هذا المسار عدة فئات من القراء:

- مستخدم لينكس الذي يريد الانتقال من تشغيل التطبيقات إلى فهم النظام الذي يشغّلها.
- مسؤول الأنظمة والعامل في التشغيل أو في فرق التطوير والتشغيل، ممن يحتاجون إلى تفسير الأعطال والتعامل مع الخدمات والموارد.
- المطوّر الذي يريد معرفة ما يقدمه نظام التشغيل لبرنامجه، بدل النظر إليه بوصفه صندوقًا مغلقًا.
- الطالب والباحث الذي يدير جهازه بنفسه ويحتاج إلى أساس موثوق للتجربة.
- الهاوي الذي يحب أن يفكك المشكلة، ويغيّر عاملًا واحدًا، ثم يلاحظ النتيجة.

لا يلزم أن تكون مبرمجًا. يكفي أن تعرف معنى الملف والمجلد، وأن تستطيع استخدام واجهة نظامك وفتح تطبيق الطرفية. ستظهر أسماء دوال مثل `fork()`، لكننا سنشرح وظيفتها دون افتراض معرفة بلغة البرمجة C.

ولا يلزم أن تفهم كل التفاصيل من القراءة الأولى. إذا استطعت في نهاية الفصل أن تشرح الفرق بين البرنامج والعملية، وبين المستخدم ذي الصلاحيات العليا والنواة، فقد بنيت أساسًا مهمًا بالفعل.

كيف تقرأ وتجرّب؟

تتدرج فصول الكتاب الأصلي من الأفكار الأساسية إلى التفاصيل. اتبع هذا التدرج: اقرأ الفكرة العامة أولًا، ثم جرّب مثالًا صغيرًا، ثم ارجع إلى التفاصيل التي تجيب عن سؤال ظهر أثناء التجربة. لا تجعل المصطلحات المتقدمة في نهاية موضوع ما تمنعك من متابعة المسار كله.

لكل جلسة تعلم قصيرة، جرّب الخطوات الآتية:

1. اختر مفهومًا واحدًا، مثل «العملية»، واكتب له تعريفًا بكلماتك.
2. توقّع ما سيفعله الأمر قبل تشغيله.
3. شغّله على جهاز تجريبي، واقرأ الناتج بدل الاكتفاء بنجاح التنفيذ.
4. قارن الناتج بتوقعك، وسجّل السؤال الذي بقي بلا إجابة.
5. أعد الشرح لشخص آخر، أو لنفسك كتابةً. إذا تعثرت في الشرح، فأنت تعرف الآن أين تحتاج إلى المراجعة.

يفضّل أن يكون أمامك نظام لينكس تستطيع التجربة عليه. يمكن أن يكون جهازًا مخصصًا للتعلم أو آلة افتراضية. احتفظ بنسخة احتياطية من الملفات المهمة، وبنقطة استعادة للآلة الافتراضية قبل التجارب التي تغيّر إعدادات النظام. هذه وسائل لتقليل المخاطر، وليست ضمانًا ضد كل خطأ.

استخدم حسابًا عاديًا في معظم الوقت. قد تحتاج في فصول لاحقة إلى صلاحيات إدارية، لكن **أوامر مختبر الفصل الأول لا تحتاج إلى** `sudo` **أو إلى حساب** `root`. لا تختبر أوامر إدارية على خادم إنتاج أو جهاز يعتمد عليه الآخرون.

> **توضيح حديث — من الإعداد العربي:** قد تختلف الصورة داخل حاوية أو بيئة مقيدة عنها في آلة افتراضية كاملة. فالحاويات تستخدم عادة نواة النظام المضيف، وقد تعرض مجموعة محدودة من العمليات والموارد. إذا اختلف ناتج أمر عن المثال، افحص طبيعة بيئتك قبل أن تفترض وجود خطأ. راجع مرجع فضاءات الأسماء في قسم المراجع.

قراءة الأوامر دون التباس

الطرفية تطبيق يعرض النص ويسمح بإدخاله، أما **الصدفة** فهي البرنامج الذي يفسّر الأوامر داخلها. الطرفية ليست النواة، والصدفة ليست النواة أيضًا. سنفصل هذه العلاقة أكثر في الفصل الثاني عند إعداده.

يستخدم الأصل الرمز `$` للدلالة على محث حساب عادي، والرمز `#` للدلالة على محث حساب ذي صلاحيات عليا. المحث ليس جزءًا من الأمر الذي تنسخه. وقد يختلف شكله على جهازك؛ لذلك لا تعتمد على شكله وحده لمعرفة صلاحياتك.

في هذا الإعداد نضع الأوامر القابلة للنسخ **من دون المحث**. مثلًا:

```
ls /
```

يعرض هذا الأمر محتويات الدليل الجذر. لا تكتب $ قبله، ولا تستبدل الشرطة المائلة بعلامة أخرى. إذا عرضنا مخرجات، فستأتي في كتلة مستقلة موسومة بأنها مخرجات أو مثال توضيحي، لا أمر مطلوب تشغيله.

نترك أسماء الأوامر والخيارات والمسارات والمتغيرات كما هي: `ls` و`/etc` و`PATH`. تظل كتل الأوامر من اليسار إلى اليمين، حتى داخل الصفحات العربية. أما الشرح المحيط بها فمن اليمين إلى اليسار. والأرقام المستخدمة في الأقسام، مثل `1.3.2`، تتبع ترقيم الأصل لتسهيل المقارنة معه.

قد ترى في المصادر أسماء توثيق مثل `fork(2)` أو `ps(1)`. الرقم بين القوسين هو قسم الدليل، وليس جزءًا من اسم الأمر، ولا رقم إصدار له. سنعرض مراجع واضحة يمكنك الرجوع إليها عندما ترغب في مزيد من التفصيل.

خريطة الكتاب والطبعة التي نعتمدها

يمكن النظر إلى الكتاب بوصفه ثلاث مراحل مترابطة:

- **بناء الصورة الأولى:** التعرف إلى مكونات النظام، ثم أدوات سطر الأوامر وبنية الأدلة.
- **تتبّع النظام من الداخل:** الأجهزة والتخزين والإقلاع والخدمات والإعداد والموارد والشبكات.
- **اكتساب مهارات العمل عليه:** سكربتات الصدفة، ونقل الملفات، وبيئات المستخدم، وسطح المكتب، وأدوات التطوير وبناء البرامج.

يحاول الأصل شرح المبادئ المشتركة بين التوزيعات، مع أمثلة من عائلتي Debian وRed Hat ومشتقاتهما. يركز على الحواسيب المكتبية والخوادم، وتنتقل كثير من الأفكار إلى أنظمة أخرى تستخدم لينكس، لكن أسماء الأدوات والقيود والتفاصيل قد تتغير.

توضح مقدمة الطبعة الثانية أن الكتاب انتقل من التركيز على بعض صعوبات التركيب والإعداد القديمة إلى اهتمام أكبر بدور النواة وإدارة الأجهزة والإقلاع. وهذا وصف للتغييرات بين طبعات الكتاب الأصلي، **لا إعلان عن إصدار عربي جديد مكتمل.**

يحمل الملف المصدر بيانات نشر من عامي 2014 و2015، وتذكر صفحة الحقوق فيه الطبعة الثانية وحقوق عام 2015. لهذا لا نتعامل مع أسماء الخدمات والتوزيعات المذكورة فيه بوصفها وصفًا مضمونًا لجهاز معاصر. نحتفظ بالمفهوم، ونفصل عنه التوضيح الحديث عندما يلزم، بدل تحديث النص بصمت أو نسب معلومات جديدة إلى المؤلف.

اصطلاحات نتفق عليها منذ البداية

نعتمد «لينكس» اسمًا شائعًا في الشرح. وعندما تكون الدقة مهمة، سنميز بين **نواة لينكس** و**التوزيعة** التي تجمع النواة مع الأدوات والمكتبات والخدمات والتطبيقات. تسمية GNU/Linux تبرز دور مشروع GNU في كثير من هذه الأنظمة؛ والتمييز بين المكونات أهم هنا من الخلاف على الاسم المختصر.

نذكر المقابل الإنجليزي للمصطلح الأساسي عند تقديمه، ثم نلتزم بالمقابل العربي قدر الإمكان. نقول «عملية» عند الحديث عن برنامج قيد التنفيذ، و«نواة» عن المكوّن المركزي لنظام التشغيل، و«نواة معالجة» عندما نقصد جزءًا من المعالج. ويجمع المعجم المصطلحات المعتمدة في هذه الدفعة.

ليس الهدف أن تنتهي من القراءة وأنت تعرف كل شيء، بل أن تعرف كيف تسأل وتلاحظ وتتحقق. ابدأ الآن بالفصل الأول: صورة صغيرة واضحة للنظام، قبل الدخول إلى تفاصيل أجزائه.

الفصل الأول: الصورة العامة

> **نطاق الفصل:** إعداد تعليمي للأقسام 1.1 إلى 1.6 من الطبعة الثانية، في صفحات ملف PDF من 8 إلى 14. يحتفظ بالترتيب المفاهيمي للأصل، ويضيف توضيحات موسومة ومختبرًا وأسئلة. أرقام الصفحات هنا هي أرقام صفحات الملف، لا أرقام طبعة ورقية أخرى.

قبل أن نبدأ

عندما تفتح متصفحًا، يبدو أنك تشغّل برنامجًا واحدًا. لكن وراء النافذة عمليات تتبادل البيانات، ومكتبات تقدم وظائف جاهزة، ونواة تنظم الوصول إلى الذاكرة والمعالج والشبكة. وقد يتصل المتصفح بخادم ويب، ويتصل ذلك الخادم بقاعدة بيانات. كيف نفهم هذا العدد من الأجزاء دون أن نضيع في تفاصيله؟

الأداة الأولى هي **التجريد**، أي اختيار مستوى مناسب من التفاصيل. عندما تركب سيارة، لا تحتاج إلى معرفة عدد البراغي التي تثبت محركها. يكفيك أن تعرف أنها تنقلك، وكيف تستعمل الباب وحزام الأمان. أما إذا كنت تقودها، فتحتاج إلى معرفة أدوات التحكم. وإذا أصبحت الرحلة شديدة الاهتزاز، فقد تفكك الصورة إلى ثلاثة احتمالات: السيارة، والطريق، وطريقة القيادة.

لم يغيّر هذا التفكيك السيارة، لكنه غيّر الطريقة التي تنظر بها إلى المشكلة. بدل اتهام المحرك مباشرة، تسأل أولًا: هل الطريق غير مستوٍ؟ وبالمثل، لا يعني بطء موقع ويب بالضرورة أن معالج حاسوبك ضعيف؛ قد يكون السبب في الاتصال أو في الخادم أو في البرنامج نفسه.

يستخدم مطوّرو البرمجيات الفكرة ذاتها. يقسمون النظام إلى **مكوّنات**، لكل منها وظيفة وواجهة للتعامل مع المكوّنات الأخرى. ليس على مطوّر تطبيق أن يعرف كل تفاصيل بطاقة الشبكة، لكنه يحتاج إلى معرفة الواجهة التي يستخدمها لإرسال البيانات.

بعد هذا الفصل، ينبغي أن تستطيع:

- وضع العتاد والنواة وعمليات المستخدم في صورة واحدة.
- التمييز بين الذاكرة الرئيسية والتخزين الدائم، وبين البرنامج والعملية.
- شرح مشاركة المعالج وعزل الذاكرة بصورة أولية.
- وصف ما يحدث عند تشغيل برنامج خارجي من الصدفة.
- التمييز بين نمط تنفيذ النواة وصلاحيات المستخدم `root`.
- ربط هذه الأفكار بملاحظات فعلية من جهاز لينكس، لا بتعريفات محفوظة فقط.

1.1 مستويات التجريد وطبقاته في نظام لينكس

لا يكفي تقسيم النظام إلى أجزاء؛ نحتاج إلى ترتيب يساعدنا على فهم العلاقات بينها. نسمّي مجموعات المكونات **طبقات** أو **مستويات** بحسب قربها من المستخدم أو من العتاد. المتصفح أقرب إلى المستخدم من برنامج تشغيل القرص، وبرنامج تشغيل القرص أقرب إلى العتاد من المتصفح.

في أبسط صورة، يتكون نظام لينكس الجاري من ثلاثة مستويات:

1. **العتاد (Hardware)**: الأجزاء المادية، مثل المعالج والذاكرة والأقراص وواجهات الشبكة. ينفّذ المعالج التعليمات ويعالج البيانات، وتحتفظ الذاكرة بما يحتاج إليه أثناء التنفيذ.
2. **النواة (Kernel)**: برنامج مركزي يدير موارد الجهاز، وينظم استخدام المعالج والذاكرة والأجهزة، ويقدم واجهات للبرامج الأخرى. هي جزء من البرمجيات، وليست قطعة إلكترونية مستقلة.
3. **فضاء المستخدم (User Space)**: البيئة التي تعمل فيها غالبية البرامج والخدمات. البرنامج أثناء تنفيذه يسمى **عملية (Process)**؛ ومجموع عمليات المستخدم يشكل الجزء الذي نتعامل معه عادة، بصورة مباشرة أو غير مباشرة.



الشكل 1-1 — مخطط تعليمي لمستويات النظام، مستند إلى فكرة الشكل في الأصل. ليست الصناديق قائمة بكل مكونات لينكس.

نمط التنفيذ ليس مجرد اسم للطبقة

توجد حدود فعلية يفرضها العتاد بين **نمط المستخدم (User Mode)** و**نمط النواة (Kernel Mode)**. في نمط المستخدم، لا تستطيع التعليمات الوصول بحرية إلى كل الذاكرة أو تنفيذ كل العمليات ذات الامتياز. وعندما تحتاج العملية إلى خدمة محمية، تطلبها من النواة عبر واجهة محددة.

في نمط النواة تتاح صلاحيات منخفضة المستوى أوسع بكثير. وهذا ضروري لإدارة الجهاز، لكنه يعني أيضًا أن خطأً في النواة أو في برنامج تشغيل يعمل ضمنها قد يؤثر في النظام كله. أما خطأ في تطبيق معزول، فكثيرًا ما ينتهي بتوقف تلك العملية وحدها بدل توقف بقية البرامج.

احفظ الفرق بين اصطلاحين متقاربين:

- **النمط** يصف مستوى الامتياز الذي ينفذ به المعالج التعليمات.
- **الفضاء** يشير هنا إلى مجال الذاكرة والبيئة التي تعمل فيها النواة أو عمليات المستخدم.

يُسمّى المجال المحمي المخصص للنواة **فضاء النواة (Kernel Space)**. أما فضاء المستخدم فليس ذاكرة واحدة مشتركة يستطيع أي تطبيق العبث بكل ما فيها؛ لكل عملية عادة مساحة عناوين خاصة بها، كما سنرى في القسم 1.3.2.

الحماية لا تلغي كل المخاطر

إذا تعطل المتصفح، يفترض ألا يتسبب ذلك وحده في إتلاف ذاكرة عملية حسابية تعمل في الخلفية. لكن عزل الذاكرة لا يحمي ملفاتك من برنامج سمحت له بتعديلها. عملية تعمل باسمك قد تحذف ملفات تملكها، وعملية ذات صلاحيات واسعة قد تُحدث ضررًا أوسع.

> **إضافة تعليمية:** هناك فرق بين «لا يستطيع البرنامج الكتابة مباشرة في ذاكرة النواة» و«لا يستطيع البرنامج إيذائي». قد يكون ممنوعًا من الأولى ومسموحًا له بتغيير بياناتك. الصلاحيات والنسخ الاحتياطية والتحديثات تكمل بعضها؛ ولا يغني واحد منها عن البقية.

1.2 العتاد: فهم الذاكرة الرئيسية

الذاكرة الرئيسية، التي نرمز إليها عادة بـRAM، مساحة سريعة نسبيًا للاحتفاظ بالتعليمات والبيانات التي يحتاج إليها النظام أثناء عمله. على مستوى مبسط جدًا، تحتوي على أنماط من الأصفار والآحاد. يسمى كل صفر أو واحد بتًا **(Bit)**، وتتكون البايتات المعتادة من ثمانية بتات.

النواة والبرامج والبيانات ليست أشياء سحرية داخل الجهاز؛ إنها تمثيلات بتية تعطيها طريقة الاستخدام معناها. قد يمثل نمط ما جزءًا من تعليمة، أو حرفًا، أو عددًا، أو لونًا في صورة. يتعامل المعالج مع التعليمات والبيانات ويغيّر حالة النظام بتنفيذ العمليات عليها.

يفرق هذا التصور بين **الذاكرة الرئيسية والتخزين الدائم**. ملفاتك على قرص أو وحدة SSD يمكن أن تبقى بعد انقطاع الطاقة، أما محتوى ذاكرة RAM المعتادة فيضيع عند انقطاعها. عندما تشغّل برنامجًا، يُحمّل ما يلزم من تعليماته وبياناته إلى الذاكرة، ولا يتحول القرص نفسه إلى معالج.

> **إضافة تعليمية:** تشبيه الذاكرة بسطح مكتب والتخزين بخزانة مفيد كبداية: تضع على المكتب ما تعمل عليه الآن، وتحتفظ في الخزانة بما تريد بقاءه. لكنه تشبيه محدود؛ فالأنظمة تستخدم التخزين المؤقت وخرائط الذاكرة وآليات أخرى، ولا تنسخ بالضرورة البرنامج كله دفعة واحدة.

ماذا نعني بحالة النظام؟

الحالة (State) وصف للنظام أو لأحد مكوناته في لحظة معينة. إذا كان لدينا أربعة بتات فقط، فكل ترتيب من الترتيبات الآتية يمثل حالة مختلفة:

```
0110
0001
1011
```

لكن وصف ملايين البتات واحدة واحدة لا يساعدنا عادة. نرفع مستوى التجريد فنقول: «العملية تنتظر إدخالًا»، أو «البرنامج يقرأ ملفًا»، أو «هذه الصفحة من الذاكرة للقراءة فقط». هذه أوصاف مفيدة لحالات لها تمثيل أدق داخل الجهاز.

قد يظهر أيضًا مصطلح **صورة العملية (Process Image)**. لا يعني صورة مرئية على الشاشة، بل تمثيل البرنامج وبياناته في الذاكرة. وسيساعدنا هذا المصطلح عند الحديث عن استبدال البرنامج الجاري بواسطة عائلة exec.

> **توضيح تقني — من الإعداد العربي:** لا يقرأ المعالج كل معلومة من شرائح RAM مباشرة في كل خطوة؛ توجد سجلات وذاكرات مخبئية وآليات لنقل البيانات. نتجاوز هذه التفاصيل الآن لأن المقصود بناء نموذج للذاكرة والعمليات، لا شرح البنية الدقيقة للمعالج.

1.3 النواة

تدير النواة موارد تتشاركها عمليات كثيرة. عليها أن تعرف أي عملية تستطيع استخدام المعالج، وأي صفحات ذاكرة مخصصة لها، وكيف تنفذ طلبًا إلى جهاز، وما إذا كان الطلب مسموحًا أصلًا.

يمكن جمع مسؤولياتها في أربعة مجالات:

المجال	السؤال الذي تجيب عنه النواة
إدارة العمليات	مَن يعمل الآن؟ ومَن ينتظر؟ وكيف نبدأ العمل أو ننهيه؟
إدارة الذاكرة	لمن تخص هذه الذاكرة؟ وما الذي يجوز مشاركته أو الوصول إليه؟
برامج تشغيل الأجهزة	كيف نطلب من عتاد معين تنفيذ عملية بطريقة مضبوطة؟
نداءات النظام والخدمات المساندة	كيف تحصل عمليات المستخدم على خدمات لا تستطيع تنفيذها مباشرة؟

هذه حدود تعليمية، وليست جدرانًا بين وظائف مستقلة تمامًا. قراءة ملف مثلًا قد تجمع بين إدارة الذاكرة، وجدولة عملية تنتظر، والتواصل مع جهاز تخزين.

1.3.1 إدارة العمليات

البرنامج تعليمات محفوظة، غالبًا في ملف. أما **العملية** فهي مثيل جارٍ من التنفيذ، له معرّف وذاكرة وحالة وموارد مرتبطة به. يمكنك تشغيل البرنامج نفسه أكثر من مرة فتظهر عمليات مختلفة، وقد يستخدم تطبيق واحد عدة عمليات.

تشمل إدارة العمليات إنشاءها، وجدولتها، وإيقافها مؤقتًا عند الحاجة، واستئنافها، وإنهاءها. الجزء الأقل وضوحًا هو مشاركة المعالج: كيف يبدو أن المتصفح ومشغل الصوت ومحرر النصوص تعمل جميعًا في الوقت نفسه؟

لنتخيل أولًا معالجًا منطقيًا واحدًا. لا تنفذ عليه مهمتان مستقلتان في اللحظة نفسها، لكن النواة تمنح كل مهمة فرصة للعمل، ثم قد تنتقل إلى أخرى. يكون التناوب سريعًا بحيث نرى النظام متعدد المهام، لا جهازًا يتوقف بعد كل تعليمة ليستأذننا.

تسمى الفترة التي نتصور أن عملية حصلت فيها على المعالج **شريحة زمنية (Time Slice)**. ويسمى الانتقال من سياق تنفيذ إلى سياق آخر **تبديل السياق (Context Switch)**. والسياق هو المعلومات اللازمة لمتابعة التنفيذ من الموضع الصحيح، ومنها قيم سجلات المعالج وحالة الجدولة ومعلومات مساحة العناوين.

> **إضافة تعليمية:** تخيل مدرسًا يساعد عدة طلاب بالتناوب. لا يجيب الجميع في اللحظة نفسها، لكنه يتذكر أين توقف مع كل طالب. هذا يشرح التزامن، لا سرعات المعالجات ولا الخوارزمية الحقيقية للجدولة.

صورة مبسطة لتبديل السياق

في سيناريو تعليمي تنتهي فيه فرصة عملية لاستخدام المعالج، يمكن تلخيص ما يحدث هكذا:

1. تحدث مقاطعة من المؤقت، فينتقل التنفيذ إلى النواة وتستعيد التحكم على ذلك المعالج.
2. تحفظ النواة حالة التنفيذ اللازمة لاستئناف العملية لاحقًا. لا يعني هذا نسخ محتويات ذاكرة العملية كلها عند كل تبديل.
3. تعالج ما يلزم من أعمال مرتبطة بالمقاطعة أو بأحداث أصبحت جاهزة.
4. يفحص المجدول المهام القابلة للتنفيذ ويختار المهمة التالية وفق سياسة الجدولة.
5. تجهّز النواة سياق المهمة المختارة وما يلزم لاستخدام مساحة عناوينها.
6. تُضبط آلية متابعة الوقت والجدولة بحسب تنفيذ النظام.
7. يعود التنفيذ إلى نمط المستخدم عندما تصبح العملية المختارة جاهزة للمتابعة فيه.

هذه **صورة تفسيرية**، لا تسلسلًا إلزاميًا ثابتًا لكل حالة. قد تترك العملية المعالج لأنها تنتظر بيانات من الشبكة قبل انتهاء أي فرصة زمنية متوقعة. وقد يختار المجدول العملية نفسها مجددًا إذا لم توجد مهمة أخرى أنسب للتنفيذ.

التزامن والتوازي

التزامن (Concurrency) يعني أن عدة أعمال تتقدم خلال الفترة نفسها، ولو بالتناوب. أما **التوازي (Parallelism)** فيعني تنفيذ أعمال فعلًا في اللحظة نفسها على موارد تنفيذ متعددة. على جهاز متعدد الأنوية يمكن أن يحدث الأمران معًا: عمل متوازٍ بين الأنوية، وتناوب بين مهام أكثر من عدد المعالجات المنطقية المتاحة.

> **توضيح حديث — من الإعداد العربي:** تستخدم الفقرة السابقة كلمة «عملية» لتبسيط الفكرة. فعليًا، تحتوي العملية على خيط تنفيذ واحد أو أكثر، وتُجدول خيوط التنفيذ بوصفها مهام قابلة للتنفيذ. تشترك خيوط العملية في مساحة عناوينها، وقد تعمل على معالجات مختلفة. ليس عدد العمليات مساويًا لعدد الأنوية. انظر مرجع `pthreads(7)`.

ولا تعمل النواة فقط بين الشرائح الزمنية. تُنفّذ تعليمات منها أيضًا لمعالجة نداءات النظام والمقاطعات والاستثناءات، مثل أخطاء الصفحات. كذلك لا يعني كل انتقال إلى نمط النواة حدوث تبديل إلى عملية أخرى؛ فقد تطلب العملية خدمة وتعود النواة إلى العملية ذاتها.

1.3.2 إدارة الذاكرة

تحتاج النواة إلى تنظيم الذاكرة بحيث تتحقق متطلبات تبدو متعارضة:

- يبقى لها مجال محمي لا تستطيع عمليات المستخدم الوصول إليه اعتباطًا.
- تحصل كل عملية على مساحة للعمل، من دون أن تكتب مباشرة في الذاكرة الخاصة بعملية أخرى.
- تستطيع عمليات مشاركة مناطق محددة عندما يكون ذلك مقصودًا ومسموحًا.
- يمكن حماية بعض المناطق من الكتابة، مثل صفحات تحتوي على تعليمات ينبغي ألا تتغير.
- يمكن استخدام التخزين المساند لبعض البيانات عند تفعيل آليات المبادلة، بدل اشتراط وجود كل محتوى الذاكرة الافتراضية في RAM طوال الوقت.

يساعد العتاد على ذلك بواسطة **وحدة إدارة الذاكرة (MMU)**. في نموذج لينكس المعتاد على الحواسيب العامة، تتعامل العملية مع **عناوين افتراضية**، لا مع أرقام مواقع مادية حرة في شرائح الذاكرة. تنظم النواة الخرائط وحقوق الوصول، وتستخدم وحدة إدارة الذاكرة هذه المعلومات عند ترجمة العناوين والتحقق من الوصول.

يمكن أن تستخدم عمليتان العنوان الافتراضي نفسه، لكنه يشير في كل منهما إلى موضع مادي مختلف. ولهذا لا يكفي أن تعرف عملية عنوانًا في برنامج آخر كي تصبح قادرة على قراءة بياناته.

فكر في المثال الآتي بوصفه رسمًا للمفهوم، لا قيمًا مأخوذة من جهاز حقيقي:

العملية	العنوان الذي تستخدمه	نتيجة الترجمة
العملية أ	`0x1000`	موضع مادي مخصص للعملية أ
العملية ب	`0x1000`	موضع مادي آخر مخصص للعملية ب

تسمى البنى المستخدمة لتنظيم هذه الخرائط **جداول الصفحات (Page Tables)**. تدير النواة هذه الجداول، ويتعاون العتاد معها على تطبيقها. وقد تشير خرائط عمليتين إلى الصفحة المادية نفسها إذا كانت المشاركة مقصودة، مثل بعض الصفحات المشتركة للقراءة فقط.

> **توضيح تقني — من الإعداد العربي:** الذاكرة الافتراضية ليست مرادفًا لمساحة المبادلة swap. يظل لكل عملية فضاء عناوين افتراضي حتى على جهاز بلا مساحة مبادلة. كما أن وجود مساحة افتراضية كبيرة لا يضمن وجود ذاكرة فعلية كافية لكل طلب؛ قد يفشل تخصيص الذاكرة أو يتعرض النظام لضغط شديد. سنحتاج إلى هذا التمييز عند دراسة الموارد في الفصل الثامن.

المطلوب الآن فهم العلاقة: **العملية تستخدم عناوين؛ النواة تدير الخرائط والسياسات؛ والعتاد يساعد على الترجمة والحماية.** لا تحتاج بعد إلى حفظ بنية جدول الصفحات أو أحجام الصفحات المختلفة.

1.3.3 برامج تشغيل الأجهزة وإدارتها

قد تؤدي بطاقتا شبكة الوظيفة نفسها للمستخدم، لكن تختلف الأوامر التي يفهمها عتاد كل منهما. لو اضطر كل تطبيق إلى دعم كل بطاقة وكل قرص مباشرة، لأصبح تطوير البرامج وصيانتها أصعب بكثير.

لهذا توجد **برامج تشغيل الأجهزة (Device Drivers)**. يعرف برنامج التشغيل تفاصيل التعامل مع جهاز أو فئة من الأجهزة، ويساعد على تقديم واجهة منتظمة لبقية النظام. يطلب التطبيق عملية أعلى مستوى، وتتولى طبقات النظام إيصال الطلب إلى العتاد المناسب.

ينتمي كثير من برامج التشغيل إلى النواة، سواء أكان مدمجًا فيها أم محمّلًا بوصفه وحدة. ويسمح هذا الموقع بالوصول المضبوط إلى موارد لا ينبغي أن تكون متاحة مباشرة لأي عملية. فالتعامل الخاطئ مع جهاز تخزين أو مع عمليات الوصول المباشر إلى الذاكرة قد يؤثر في بيانات غير البرنامج الذي بدأ الطلب.

> **إضافة تعليمية:** لا تفهم عبارة «برنامج تشغيل» بمعنى تطبيق تفتحه من قائمة البرامج. وليس كل برنامج يتعامل مع جهاز جزءًا من النواة؛ توجد أدوات ومكتبات وخدمات في فضاء المستخدم تتعاون مع برامج التشغيل. تفاصيل هذا التعاون تأتي في فصل الأجهزة.

الفكرة المهمة أن البرنامج لا يحتاج عادة إلى معرفة الشركة المصنعة للقرص لكي يقرأ ملفًا. يحصل على واجهة مناسبة، بينما تختبئ تفاصيل كثيرة وراءها. هذا هو التجريد نفسه الذي بدأنا به الفصل، ولكن في مسار عملي داخل النظام.

1.3.4 نداءات النظام والخدمات المساندة

كيف تطلب عملية في نمط المستخدم خدمة من النواة؟ تستخدم **نداء نظام (System Call)**، مباشرة أو عبر دالة في مكتبة. فتح ملف، وقراءة بيانات، وكتابة بيانات، وبدء عملية أو انتظارها أمثلة على أعمال تتضمن التواصل مع النواة.

ليس نداء النظام أمرًا تكتبه بالضرورة في الطرفية. عندما تنفذ أمرًا مثل `ls`، قد يستخدم البرنامج عدة نداءات نظام أثناء عمله. القوسان في أسماء مثل `fork()` أسلوب شائع لكتابة أسماء الدوال؛ لا يعنيان أنك مطالب بإدخال الاسم وحده في الصدفة.

إنشاء عملية ليس هو استبدال برنامجها

لفهم تشغيل البرامج، نحتاج إلى التمييز بين عمليتين مفاهيميتين:

- `fork()` — **إنشاء عملية ابنة:** تنشئ عملية موجودة عملية جديدة تشبهها عند البداية. تسمى الأولى العملية الأم، والثانية العملية الابنة. يحصل الابنة على معرّف عملية خاص به، وله مساحة عناوين مستقلة من حيث المعنى الذي يراه البرنامج.
- عائلة `exec` — **استبدال البرنامج الجاري:** تستبدل العملية برنامجها الحالي ببرنامج آخر. لا تُنشأ بسبب هذا الاستبدال وحده عملية جديدة ذات معرّف جديد؛ يستمر معرّف العملية بينما تتغير صورة البرنامج وذاكرته المرتبطة به.

> **توضيح تقني — من الإعداد العربي:** يعبّر الاسم المختصر `exec()` في الشرح عن عائلة واجهات، وليس اسم نداء وحيد يكفي لاستدعائه حرفيًا. تنتهي واجهات شائعة من هذه العائلة إلى نداء مثل `execve()`. وعند نجاح الاستبدال لا يعود التنفيذ إلى البرنامج السابق بعد النداء. انظر مرجع `execve(2)`.

لنفترض أنك تستخدم صدفة تفاعلية، ثم تشغّل البرنامج الخارجي الآتي:

```
ls /
```

في النموذج المعتاد المبسط، تنشئ الصدفة عملية ابنة، ثم يستبدل الابنة صورته ببرنامج `ls`. تعرض العملية الجديدة محتويات الدليل وتنتهي، بينما تبقى الصدفة الأم لتستقبل أمرًا آخر. قد تنتظر الصدفة انتهاء الأمر الأمامي قبل أن تعرض محثها من جديد.

قبل التشغيل
صدفة أم — معرّف افتراضي 1200

`fork()` ينشئ عملية ابنة

العملية الأم	العملية الابنة
تبقى صدفة، ومعرّفها 1200	تبدأ كنسخة من الصدفة، بمعرّف 1201
يمكنها انتظار انتهاء الابنة	تستخدم إحدى واجهات exec لتشغيل ls
تعرض المحث مجددًا بعد انتهاء الأمر الأمامي	تبقى بالمعرّف 1201، ثم تنتهي بعد إنجاز العمل

الشكل 2-1 — مخطط تعليمي لإنشاء عملية ابنة ثم استبدال برنامجها. الأرقام للتوضيح فقط؛ لا تستخدمها لاستهداف عمليات على جهازك.

كلمة «نسخة» في شرح `fork()` لا تعني نسخ كل صفحات الذاكرة المادية فورًا. يستخدم لينكس **النسخ عند الكتابة (Copy-on-Write)**: يمكن أن تتشارك الصفحات المادية في البداية، وتُنشأ نسخة مستقلة عند الحاجة إلى تعديل صفحة خاصة مشتركة بهذه الطريقة. تظل العزلة المنطقية بين فضاءي العناوين قائمة. يشرح مرجع `fork(2)` هذه الآلية.

هناك أيضًا حدود لهذا النموذج التعليمي. بعض أوامر الصدفة مدمج فيها، فلا تحتاج إلى تشغيل برنامج خارجي جديد. وتوجد واجهات وآليات مثل `clone()` و`clone3()` تتحكم في إنشاء المهام وما تتشاركه؛ لذلك لا نحول مثال `fork` ثم `exec` إلى قاعدة حرفية تصف كل مسار تنفيذ على لينكس. تظل الفكرة التي يجب تذكرها صحيحة: **الإنشاء يضيف عملية، والاستبدال يغيّر البرنامج داخل عملية موجودة.**

توجد عند بدء النظام عملية أولى في فضاء المستخدم، تؤدي دور التهيئة وتكون ذات المعرّف `1` في فضاء معرّفات العمليات الرئيسي. يقدمها الأصل باسم `init`، وسنعود إلى بدء فضاء المستخدم في الفصل السادس. لا تستنتج من هذا الاسم أن كل جهاز يستخدم البرنامج نفسه لتنفيذ هذا الدور.

أجهزة موجودة بوصفها واجهات برمجية

تقدم النواة أيضًا **أجهزة زائفة (Pseudo-devices)**: واجهات تبدو للبرنامج شبيهة بملفات الأجهزة، لكنها لا تقابل بالضرورة قطعة عتاد مستقلة. من أمثلتها `/dev/random`، الذي يتيح الوصول إلى بيانات من مولّد الأرقام العشوائية في النواة.

يسمح هذا التصميم بإعادة استخدام واجهات مألوفة، مثل فتح ملف والقراءة منه، للوصول إلى وظيفة يقدمها النظام. وكون الجهاز زائفًا لا يعني تجاوز النواة؛ ففتح واجهته وقراءتها يظلان يتضمنان نداءات نظام.

> **إضافة تعليمية:** المثال هنا لشرح الواجهة، وليس وصفة لاختيار مصدر عشوائية لتطبيق أمني. لا تصمم التشفير أو توليد المفاتيح بنفسك اعتمادًا على هذا الوصف المختصر؛ استخدم واجهات ومكتبات موثقة مناسبة للمهمة.

1.4 فضاء المستخدم

معظم العمل الذي تراه يحدث في فضاء المستخدم: تحرير النصوص، وتصفح الويب، وخدمة قواعد البيانات، وإدارة جلسة سطح المكتب. وقد تسمع الاسم الإنجليزي `userland` للدلالة على البيئة نفسها بصورة غير رسمية.

لا تعني **عملية مستخدم** بالضرورة أن شخصًا يتفاعل معها الآن. خادم الويب الذي يعمل في الخلفية عملية مستخدم أيضًا. ولا تعني العبارة أنها تعمل باسم مستخدم عادي؛ يمكن أن تعمل خدمة باسم `root` وتبقى عملية في فضاء المستخدم.

يمكن ترتيب مكونات هذه البيئة، لأغراض الفهم، في مستويات خدمة تقريبية. توجد أدوات صغيرة تؤدي وظائف محددة، وخدمات أكبر تعتمد عليها، وتطبيقات يتعامل معها المستخدم مباشرة.



الشكل 3-1 — ترتيب تعليمي تقريبي لخدمات فضاء المستخدم. جميع الصناديق هنا في فضاء المستخدم، بما فيها الصندوق السفلي.

هذا الرسم ليس قانونًا يفرضه النظام. قد يكتب برنامج سجلاته بنفسه، أو يرسلها إلى خدمة تسجيل مشتركة. ويمكن اعتبار خادم قاعدة البيانات تطبيقًا عالي المستوى لأن عمله معقد، أو خدمة وسطية لأن تطبيقًا آخر يعتمد عليه. التصنيف يتغير بحسب السؤال الذي نحاول الإجابة عنه.

يشير الأصل إلى `syslog` مثالًا على خدمة تسجيل مشتركة. **ومن التوضيحات المضافة:** قد ترى أيضًا `systemd-journald` في أنظمة تستخدم منظومة `systemd`. ليس المطلوب الآن اختيار خدمة تسجيل، بل فهم أن مكوّنًا يستطيع تفويض وظيفة إلى مكوّن آخر. وتظل تفاصيل النظام الفعلية موضوعًا للفحص، لا استنتاجًا من الرسم وحده.

> **إضافة تعليمية:** الفصل بين طبقات الخدمة في هذا القسم تنظيمي ووصفي. أما الحد بين نمط المستخدم ونمط النواة في القسم 1.1 فهو حد امتياز يفرضه العتاد والنظام. لا تخلط بين نوعي «الطبقات».

1.5 المستخدمون

في لينكس، **المستخدم (User)** هوية يمكن أن تُنسب إليها عمليات وملفات وصلاحيات. قد تمثل شخصًا حقيقيًا، وقد تكون حسابًا مخصصًا لخدمة. وجود حساب على النظام لا يعني بالضرورة وجود شخص يجلس أمام شاشة ويسجل الدخول به.

تتعامل عادة مع اسم مفهوم، مثل `amina`. أما النواة فتستند في كثير من قرارات الصلاحيات إلى معرّفات رقمية، من أهمها **معرّف المستخدم (UID)**. توجد آليات في فضاء المستخدم تربط الاسم بالرقم؛ وسنعود إلى تفاصيل حسابات المستخدمين في الفصل السابع.

لعمليات المستخدم بيانات اعتماد تحدد الهوية والصلاحيات التي تعمل بها. وفي النموذج الأساسي، يستطيع المستخدم إدارة عملياته والوصول إلى ملفاته وفق الأذونات، ولا يملك حرية التصرف في عمليات الآخرين وملفاتهم. توجد تفاصيل واستثناءات، لكن الحدود بين الحسابات أحد أسباب وجود مفهوم المستخدم أصلًا.

المستخدم ذو الصلاحيات العليا

يمثل الحساب `root`، المرتبط تقليديًا بالمعرّف `0`، المستخدم ذا الصلاحيات العليا **(Superuser)**. تسمح امتيازاته الإدارية بأعمال تتجاوز ما يسمح به للحساب العادي، مثل إدارة كثير من عمليات المستخدمين وإعدادات النظام.

لكن هناك حقيقة لا تتغير بسبب قوة الحساب:

البرنامج الذي يعمل باسم `root` لا يصبح هو النواة، ولا ينفذ كل تعليماته في نمط النواة. يبقى برنامجًا في فضاء المستخدم، ويطلب الخدمات من النواة، لكنها تسمح له بطلبات أكثر بناءً على بيانات اعتماده والسياسات المطبقة.

> **توضيح حديث — من الإعداد العربي:** لا تختزل صلاحيات لينكس المعاصر في عبارة «root يستطيع كل شيء». تقسم آلية القدرات `capabilities` امتيازات واسعة إلى صلاحيات أصغر، وقد تضيف فضاءات الأسماء وسياسات الحماية وقيود أخرى حدودًا إضافية. والحساب المسمى `root` داخل بيئة معزولة ليس بالضرورة مكافئًا لصلاحيات مدير النظام المضيف. راجع مرجعي القدرات وفضاءات الأسماء.

استخدم أقل صلاحيات تحقق المهمة. الخطأ في اسم مسار أثناء العمل بصلاحيات واسعة قد يُنفّذ بنجاح بدل أن يمنعه النظام. لهذا نفضل التعلم بحساب عادي، ولا نضيف `sudo` إلى أمر لمجرد أن نتيجته لم توافق توقعاتنا.

ولا تخلط بين اسم المستخدم `root` و**الدليل الجذر** `/`. الأول حساب وهوية، والثاني نقطة بداية شجرة الملفات. تشابه التسمية لا يجعلهما الشيء نفسه.

المجموعات

المجموعة (Group) وسيلة لجمع مستخدمين ضمن هوية مشتركة لأغراض الصلاحيات. بدل السماح لكل مستخدم منفردًا بالوصول إلى ملف تعاوني، يمكن منح الإذن لمجموعة مناسبة وإضافة المستخدمين إليها.

ترتبط المجموعات أيضًا بمعرّفات رقمية تسمى GID. وقد ينتمي المستخدم إلى أكثر من مجموعة. سنرى تفاصيل ملكية الملفات وأذونات المجموعات لاحقًا؛ يكفي الآن أن تفهم أنها أداة لتنظيم المشاركة، لا إلغاء الحدود بين الحسابات.

1.6 إلى أين نتجه بعد ذلك؟

لدينا الآن نموذج أولي لنظام يعمل: عتاد يقدم موارد، ونواة تنظمها، وعمليات في فضاء المستخدم تنجز الأعمال. توجد النواة وأجزاء التنفيذ اللازمة للعمليات في الذاكرة، وتؤثر هويات المستخدمين في ما يجوز للعمليات طلبه.

بقي عنصر مهم لم نفصله: أين تبقى البرامج والبيانات عندما لا تكون قيد التنفيذ؟ هنا تأتي الملفات والأدلة والتخزين الدائم. ينتقل الفصل الثاني من الأصل إلى الأدوات الأساسية للتعامل مع هذه البيئة، قبل التعمق في الأجهزة والأقراص والإقلاع.

الفصل الثاني متاح الآن في هذه الدفعة التراكمية. قبل الانتقال إليه، استخدم المختبر الآتي لتحويل مفاهيم الفصل الأول إلى أشياء تستطيع ملاحظتها. لا تنتظر أن تفهم كل خيارات الأوامر؛ سنشرح وظيفة الخيارات المستخدمة هنا فقط.

مختبر عملي: شاهد الصورة على جهازك

> **إضافة تعليمية بالكامل.** هذه التجارب من الإعداد العربي وليست ترجمة لتمارين في الأصل. شغّلها بحساب عادي على نظام لينكس تجريبي. لا تغيّر إعدادات النظام ولا تحتاج إلى صلاحيات إدارية. تحتاج أوامر ps وfree إلى أدوات procps المعتادة؛ قد تكون غائبة أو مختلفة في الأنظمة المصغرة والحاويات. إذا لم يوجد أمر، اقرأ رسالة الخطأ ولا تحاول إصلاح ذلك بصلاحيات واسعة دون فهم.

التجربة الأولى: النواة ليست التوزيعة

اعرض إصدار النواة:

```
uname -r
```

ثم اقرأ معلومات التوزيعة إن كان الملف موجودًا:

```
cat /etc/os-release
```

الأمر الأول يسأل عن إصدار النواة الجارية. والثاني يقرأ ملفًا نصيًا يصف نظام التشغيل أو التوزيعة في بيئتك. قد تستخدم توزيعتان مختلفتان إصدار نواة متقاربًا، وقد تُحدَّث النواة دون أن يتغير اسم التوزيعة.

سؤال الملاحظة: هل ظهر الاسم الذي تستعمله لتوزيعتك في ناتج الأمر الأول أم الثاني؟ وإذا كنت داخل حاوية، فهل من المعقول أن تصف النتيجتان جانبين مختلفين من البيئة؟

التجربة الثانية: العمليات ومعرّفاتها

اعرض بضعة أسطر من قائمة العمليات:

```
ps -eo pid,ppid,user,comm | head -n 8
```

تعني `-e` اختيار كل العمليات التي تستطيع الأداة رؤيتها، وتحدد `-o` الأعمدة المطلوبة. يحمل العمود `PID` معرّف العملية، و`PPID` معرّف العملية الأم، و`USER` المستخدم، و`COMMAND` اسم الأمر. تنقل العلامة `|` الناتج إلى `head` ليعرض أول ثمانية أسطر، ومنها سطر العناوين.

القائمة لقطة لحظة التنفيذ، وليست شاشة متابعة مستمرة. قد تظهر فيها أداة `ps` نفسها، وقد تختلف العمليات والأرقام في كل مرة.

انظر الآن إلى العملية ذات المعرّف `1` في بيئتك:

```
ps -p 1 -o pid,comm,args
```

تختار `-p 1` ذلك المعرّف، ويضيف `args` سطر الأمر بما فيه وسائطه. قد ترى `systemd` أو برنامجًا آخر، وفي الحاوية قد ترى برنامج الحاوية نفسه. سجّل ما رأيت فعلًا، ولا تغيّر الإعداد لتجعل الناتج يطابق اسمًا في كتاب.

التجربة الثالثة: عملية جديدة دون تغيير ملفات

الصق الكتلة التالية كاملة في صدفة تدعم صياغة الصدفة التقليدية، مثل `bash` أو `dash`:

```
sleep 5 &
demo_pid=$!
ps -o pid,ppid,user,stat,comm -p "$demo_pid"
wait "$demo_pid"
```

- ينشئ `sleep 5` برنامجًا ينتظر نحو خمس ثوانٍ ثم ينتهي.
- تجعل `&` الأمر يعمل في الخلفية، حتى تستطيع الصدفة متابعة السطر التالي.

- تحفظ `$!` معرّف آخر عملية شغّلتها الصدفة في الخلفية، ونضعه في المتغير `demo_pid`.
- يعرض `ps` معلومات هذه العملية، ومنها `STAT`، أي مؤشرات حالة العملية.
- ينتظر `wait` انتهاء العملية الابنة. لا تبقى هذه التجربة تعمل إلى أجل غير محدد.

قد ترى حالة تبدأ بالحرف `S` لأن العملية نائمة انتظارًا لانقضاء المدة، لا لأنها تستهلك المعالج في عدّ متواصل. إذا تأخرت بين الأسطر، فقد تنتهي العملية قبل أن يعرضها `ps`، فتظهر العناوين فقط. هذا سلوك متوقع، لا تلف في النظام.

سؤال الملاحظة: لماذا يختلف معرّف هذه العملية عن معرّف الصدفة التي بدأتها؟ وما الذي يدل عليه عمود العملية الأم؟

التجربة الرابعة: الذاكرة والمبادلة

```
free -h
```

تعرض `-h` القيم بوحدات أسهل للقراءة. ابحث عن سطر الذاكرة وسطر المبادلة، ثم عن الأعمدة التي تعرضها نسختك من الأداة. إذا وجدت عمود `available`، فهو تقدير للذاكرة المتاحة لبدء أعمال جديدة دون الحاجة إلى المبادلة، وليس مرادفًا مباشرًا لكل الذاكرة غير المستخدمة الآن.

قد تكون مساحة المبادلة صفرًا مع استمرار النظام في استخدام الذاكرة الافتراضية؛ لا تعارض بين الأمرين. ولا تفترض أن الأرقام داخل حاوية تصف دائمًا حد الذاكرة المسموح به للحاوية نفسها؛ قد تحتاج حدود الموارد إلى أدوات وفحص إضافيين في فصل الموارد.

التجربة الخامسة: مَن أنت بالنسبة إلى النظام؟

```
id
```

يعرض الأمر معرّف المستخدم والمجموعة والمجموعات الإضافية. مثال توضيحي، **وليس ناتجًا مطلوبًا على جهازك**:

```
uid=1000(amina) gid=1000(amina) groups=1000(amina),27(sudo)
```

الأرقام والأسماء تختلف بين الأنظمة. العضوية في مجموعة اسمها `sudo` لا تعني أن كل عملية تشغلها تعمل تلقائيًا باسم `root`؛ السماح برفع الصلاحيات تحدده سياسة النظام، ويتم استخدامه عند طلب العملية الإدارية المناسبة.

لا تنشر نواتج تحتوي على معلومات شخصية أو أسماء أجهزة أو وسائط أوامر حساسة دون مراجعتها. تكفي ملاحظاتك المحلية لإنجاز هذه التجارب.

اختبر فهمك

حاول الإجابة دون الرجوع إلى الفقرات، ثم قارن شرحك بالإجابات المقترحة.

1. لماذا لا يكون تعطل المتصفح دليلًا على تعطل النواة؟ وهل يحمي عزل الذاكرة ملفاتك من كل برنامج؟
2. هل خادم يعمل بلا نافذة يُعد عملية في فضاء المستخدم؟ علّل.
3. كيف يمكن أن يعمل عشرون برنامجًا على جهاز يملك أربعة معالجات منطقية؟
4. تستخدم عمليتان العنوان الافتراضي نفسه. هل يعني ذلك أنهما تقرآن البيانات نفسها؟
5. ما الفرق الجوهري بين `fork()` وإحدى دوال عائلة `exec`؟ ماذا يحدث لمعرّف العملية في كل حالة؟
6. هل تشغيل برنامج باسم `root` يجعله ينفذ كل تعليماته في نمط النواة؟
7. لم تُفعّل مساحة مبادلة على جهازك. هل يعني ذلك عدم وجود ذاكرة افتراضية؟
8. لماذا يمكن أن يكون `uname -r` و`cat /etc/os-release` مفيدين معًا عند وصف مشكلة؟

إجابات مقترحة

1. المتصفح عملية أو مجموعة عمليات معزولة نسبيًا؛ تستطيع النواة إنهاء العملية المتعطلة دون أن تتوقف هي. لكن العملية قد تملك إذن تعديل ملفاتك، فلا يساوي عزل الذاكرة حماية البيانات من كل فعل مسموح.
2. نعم. «المستخدم» في اصطلاح فضاء المستخدم لا يشترط وجود تفاعل بشري مباشر أو نافذة. المقصود بيئة التنفيذ خارج فضاء النواة.
3. تمنح الجدولة المهام فرصًا للتنفيذ بالتناوب، ويمكن لأربع مهام مثلًا أن تنفذ بالتوازي على أربعة معالجات منطقية. كما أن كثيرًا من المهام ينتظر إدخالًا أو حدثًا ولا يحتاج إلى المعالج طوال الوقت.
4. لا. يُفسر العنوان في سياق مساحة عناوين كل عملية. قد يترجم إلى صفحتين ماديتين مختلفتين، ولا تحدث المشاركة إلا حين تُعد الخرائط لذلك.
5. ينشئ `fork()` عملية ابنة ذات معرّف جديد. أما `exec` فيستبدل البرنامج الجاري داخل العملية ويحافظ على معرّفها. الإنشاء والاستبدال مفهومان مختلفان حتى لو اجتمعا في مسار تشغيل واحد.
6. لا. تعمل تعليمات البرنامج العادية في نمط المستخدم، وتُستخدم نداءات النظام لطلب خدمات النواة. صلاحيات الحساب تؤثر في ما تسمح به النواة، ولا تلغي الفصل بين نمطي التنفيذ.

7. لا. الذاكرة الافتراضية آلية عناوين وخرائط وحماية؛ المبادلة وسيلة إضافية لإدارة بعض محتوى الذاكرة باستخدام تخزين مساند. يمكن استخدام الأولى دون الثانية.
8. يصف الأول النواة الجارية، ويصف الثاني التوزيعة أو بيئة نظام التشغيل التي يحتوي ملفها على تلك المعلومات. الجمع بينهما أدق من الاكتفاء باسم تجاري أو برقم واحد، خصوصًا في البيئات المعزولة.

خلاصة الفصل

- **التجريد** يساعدك على اختيار مستوى التفاصيل الذي يفسر المشكلة.
- **العتاد** يوفر الموارد، و**النواة** تنظم استخدامها، و**عمليات المستخدم** تنجز معظم الأعمال التي تهمك مباشرة.
- **الجدولة** تنظم التنفيذ، و**الذاكرة الافتراضية** تنظم العناوين والعزل والمشاركة.
- **نداءات النظام** طريق مضبوط لطلب خدمات النواة؛ لا يجعل استخدامها التطبيق جزءًا من النواة.
- **هوية المستخدم وصلاحياته** ليستا نمط تنفيذ المعالج. حتى `root` لا يساوي النواة.
- أفضل اختبار لفهمك أن تربط هذه الصورة بما شاهدته في `ps` و`free` و`id`، وأن تستطيع تفسير حدود كل ملاحظة.

ارجع إلى المعجم عند التباس المصطلحات، وإلى ملاحظات الإعداد للمصادر وحدود التبسيط. ويمكنك الآن متابعة الفصل الثاني: الأوامر الأساسية وبنية الأدلة.

الفصل الثاني: الأوامر الأساسية وبنية الأدلة

> **نطاق الفصل:** إعداد تعليمي للفصل الثاني كاملًا من حيث أقسامه وموضوعاته، في صفحات ملف المصدر من 15 إلى 41. نحافظ على الأقسام 2.1 إلى 2.21 وأقسامها الفرعية، وعلى أرقام الجداول والأشكال. الأمثلة المعاد تنظيمها والمختبر والتوضيحات الموسومة من الإعداد العربي؛ ليست هذه ترجمة حرفية ولا تعليمات لتعديل جهاز إنتاج.

من الصورة العامة إلى التجربة

عرفنا في الفصل الأول أن العمليات تعمل في فضاء المستخدم، وأن النواة تنظم الموارد. سنستخدم الآن أدوات تتيح لنا رؤية تلك البيئة والتعامل معها: كيف نبدأ برنامجًا؟ كيف نقرأ ملفًا ونبحث فيه؟ كيف نعرف الدليل الذي نعمل داخله؟ وماذا تعني رسالة رفض الإذن؟

يعرض الأصل أوامر من تقاليد يونكس، لأن كثيرًا من الأفكار ينتقل بين لينكس وأنظمة شبيهة بيونكس. لكن انتقال الفكرة لا يعني تطابق جميع الخيارات والمخرجات. سنفترض في الأمثلة نظام لينكس بأدوات GNU المعتادة وصدفة bash، ونذكر مواضع الاختلاف المهمة بدل الادعاء بأن كل جهاز سيعرض النص نفسه.

هذا فصل مرجعي وتطبيقي طويل نسبيًا. اقرأ الأقسام الأولى بالتتابع، ثم جرّب أمرًا أو اثنين في كل جلسة. يمكنك العودة لاحقًا إلى جداول المفاتيح والأذونات والأدلة. ليس مطلوبًا حفظ كل خيار قبل أن تبدأ العمل.

قبل نسخ أي أمر

إضافة تعليمية: استخدم حسابًا عاديًا. أنشئ مساحة مؤقتة خاصة للتجربة بدل العمل على مستنداتك أو داخل مستودع مشروع مهم:

```
lab_dir=$(mktemp -d)
printf '%s\n' "$lab_dir"
cd "$lab_dir"
pwd
```

ينشئ `mktemp -d` دليلًا جديدًا باسم غير متوقع، ونحفظ مساره في `lab_dir`. نستخدم علامات الاقتباس حول المتغير كي يبقى المسار وسيطًا واحدًا. إذا فشل الإنشاء أو الانتقال، **توقف ولا تنفّذ أمثلة الكتابة والحذف في دليل آخر.** الملفات هنا للتجربة، ولا نعتمد على بقائها بعد تنظيف الملفات المؤقتة أو إعادة التشغيل.

لأمثلة الملفات الأولى، جهّز بيانات صغيرة:

```
printf '%s\n' 'alpha' 'beta' 'alpha' > file1
printf '%s\n' 'beta' 'gamma' > file2
mkdir dir
```

سنشرح `printf` وإعادة التوجيه تدريجيًا. لا تنفّذ الفصل كله كسكربت واحد: بعض الأمثلة مستقلة، وبعضها ينتج خطأ مقصودًا، وبعض الأوامر تفاعلي أو إداري وموسوم بذلك. الأسماء العامة مثل `file` و`target` في **كل الصيغة** أمثلة لأسماء تختارها، وليست ملفات مضمونة الوجود. ولا تكتب محث `$` أو `#` قبل الأمر؛ نحذفه من الكتل القابلة للنسخ.

2.1 صدفة بورن: المسار `/bin/sh`

الصدفة برنامج يقرأ الأوامر ويفسرها ويشغّل ما يلزم لتنفيذها. وهي أيضًا بيئة برمجة صغيرة: تستطيع ربط أدوات، وتخزين قيم في متغيرات، وتنفيذ سلسلة من الأوامر. يجمع **سكربت الصدفة** تلك الأوامر في ملف نصي، وسنخصص الفصل الحادي عشر لهذا الجانب عند إعداده.

ترجع صياغة كثير من الأوامر الشائعة إلى صدفة بورن التي طُوّرت في مختبرات Bell. وتحافظ صدف حديثة على قدر كبير من تلك اللغة. أما `bash`، أي Bourne-again shell، فتقدم هذه الأساسيات وإضافات تساعد في الاستخدام التفاعلي والبرمجة.

> **توضيح حديث:** ليس `/bin/sh` اسمًا آخر مضمونًا لـ`bash`. إنه مسار لصدفة توفر اللغة المطلوبة لسكربتات النظام؛ وقد يشير إلى `dash` على أنظمة مثل Debian أو إلى تنفيذ آخر. وحتى `bash` قد تغير بعض سلوكها عند استدعائها باسم `sh`. لا تغيّر الرابط `/bin/sh` لكي يطابق الكتاب، ولا تفترض أن إضافات Bash تعمل في كل سكربت يبدأ بـ`#!/bin/sh`. راجع سياسة Debian ودليل Bash في المراجع.

يمكن تشغيل جلسة Bash بكتابة `bash` إذا كانت مثبتة؛ لا يلزم تغيير صدفة تسجيل الدخول لتجربة الفصل. وللفحص دون تغيير إعداداتك:

```
command -v bash
ls -l /bin/sh
```

الأول يبحث عن طريقة الوصول إلى Bash، والثاني يعرض معلومات مسار `sh` وقد يبيّن هدفه إن كان رابطًا رمزيًا. هذان أمران للملاحظة، لا وصفة لإصلاح أي اختلاف.

2.2 استخدام الصدفة

استخدم الحساب العادي الذي أعددته لنفسك، لا جلسة مدير النظام. لا تحتاج إلى root لتعلم التنقل وقراءة الملفات وإدارة ملفات تجريبية تملكها. سيأتي شرح الصلاحيات الإدارية في القسم 2.20 مع حدود استخدامها.

2.2.1 نافذة الصدفة

افتح تطبيق الطرفية في واجهتك الرسومية، أو استخدم جلسة نصية متاحة لك. الطرفية تعرض النص وتنقل إدخال لوحة المفاتيح؛ والصدفة التي تعمل داخلها تفسر الأوامر. اختلاف اسم الطرفية أو سطح المكتب لا يغير هذا الفصل بين الدورين.

يظهر عادة محث يشير إلى المستخدم والجهاز والدليل الحالي. كثير من المحثات ينتهي بـ$ للحساب العادي أو # للمدير، لكن شكله قابل للتعديل؛ استخدم id إذا أردت التحقق من الهوية بدل الاعتماد على الزخرفة.

ابدأ بأمر بسيط:

```
echo Hello there.
```

المخرجات المتوقعة لهذا المثال:

```
Hello there.
```

ثم اقرأ ملفًا نصيًا من النظام:

```
cat /etc/passwd
```

يعرض الأمر معلومات حسابات محلية. **توضيح تقني:** اسم الملف تاريخي؛ في الأنظمة المعتادة لا يحتوي على كلمات المرور بصورتها الصريحة، وتُحفظ عادة تجزئات كلمات المرور في ملف مقيد مثل /etc/shadow. لا تحاول فتح ذلك الملف بصلاحيات أعلى من أجل هذا التمرين، ولا تنشر بيانات حسابات جهازك دون مراجعتها.

2.2.2 الأمر `cat`

يقرأ `cat` ملفًا أو عدة ملفات، ويكتب محتواها بالتتابع إلى خرجه. تتضح تسميته، المشتقة من concatenation أي الوصل، عندما تعطيه ملفين:

```
cat file1 file2
```

إذا استخدمت بيانات التجربة السابقة، ترى أسطر الملف الأول ثم أسطر الثاني. لا يضيف `cat` فواصل سحرية بين الملفات، ولا يحوّل صيغها؛ ولهذا نستعمله هنا مع ملفات نصية معروفة، لا ملفات ثنائية قد تملأ الطرفية بمحارف مربكة.

الصيغة العامة هي `cat file1 file2 ...`؛ والنقاط تعني إمكان إضافة أسماء أخرى، لا أنها وسيط يجب نسخه حرفيًا.

2.2.3 الإدخال القياسي والإخراج القياسي

لعمليات يونكس قنوات مألوفة لنقل البيانات. يسمى المصدر الافتراضي للإدخال **الإدخال القياسي** `stdin`، وتسمى الوجهة الافتراضية للنتائج **الإخراج القياسي** `stdout`. لا يشترط أن يكون أي منهما الطرفية؛ يمكن ربطهما بملف أو بعملية أخرى.

شغّل `cat` دون أسماء ملفات:

```
cat
```

لن يعود المحث مباشرة، لأن `cat` ينتظر من الإدخال القياسي. اكتب سطرًا واضغط Enter؛ ستشاهد السطر الذي عرضته الطرفية أثناء الكتابة ثم النسخة التي كتبها `cat`. لإنهاء الإدخال، اضغط **Ctrl-D على سطر فارغ**.

> **توضيح تقني:** تعني Ctrl-D هنا إنهاء الإدخال المتاح بالطريقة التي تتعامل بها الطرفية مع نهاية الملف، ليست إشارة قتل. أما Ctrl-C فيرسل عادة إشارة `SIGINT` إلى مجموعة العمليات الأمامية. كثير من البرامج ينتهي بسببها، لكن يمكن لبرنامج أن يعالجها بطريقة أخرى. لا تخلط بين نهاية البيانات ومقاطعة التنفيذ.

عندما لا تحدد ملفًا، تقرأ أدوات كثيرة من `stdin`، ثم تكتب نتائجها إلى `stdout`. توجد قناة ثالثة للتشخيص اسمها **الإخراج القياسي للأخطاء** `stderr`؛ سنستخدمها في القسم 2.14.1. هذه القنوات من أسباب سهولة تركيب أدوات صغيرة لحل مسائل أكبر.

2.3 أوامر أساسية

يتكون سطر الأمر من اسم الأمر، ثم **وسائط** قد تكون خيارات أو أسماء ملفات أو قيمًا. تغير الخيارات طريقة العمل؛ مثل `-l` لعرض معلومات أطول. لا تتطابق الخيارات بين كل الأدوات، ولذلك نعود إلى توثيق الأداة بدل تخمين معناها.

2.3.1 الأمر `ls`

يعرض `ls` أسماء محتويات الدليل الحالي إذا لم تحدد مسارًا. جرّب:

```
ls
ls -l
ls -F
```

تعرض `-l` قائمة مفصلة، وتضيف `-F` علامة تساعد على تمييز بعض الأنواع، مثل `/` بعد اسم دليل أو `*` بعد ملف قابل للتنفيذ. العلامات المعروضة ليست جزءًا من أسماء الملفات نفسها.

مثال توضيحي مختصر لقائمة طويلة، وليس وصفًا لجهازك:

```
-rw-r--r-- 1 amina users   17 Sep  7 10:00 file1
drwxr-xr-x 2 amina users 4096 Sep  7 10:00 dir
```

من اليسار إلى اليمين: نوع الملف وأذوناته، وعدد الروابط الصلبة، والمالك، والمجموعة، والحجم بالبايت عادة، ووقت التعديل، ثم الاسم. حجم الدليل الظاهر ليس مجموع أحجام كل الملفات داخله. وقد تتغير الأعمدة الإضافية والتواريخ والألوان بحسب النسخة والخيارات والإعداد المحلي. سنفكك سلسلة الأذونات في القسم 2.17.

2.3.2 الأمر `cp`

ينسخ `cp` ملفًا إلى اسم آخر. داخل دليل التجربة:

```
cp file1 copy.txt
```

يبقى `file1` وتُنشأ نسخة باسم `copy.txt`. إذا كانت الوجهة ملفًا موجودًا فقد يُستبدل محتواه، فلا تتعامل مع النسخ كعملية لا يمكنها إتلاف بيانات. يمكنك طلب تأكيد عند الاستبدال بواسطة `cp -i`، لكن الوقاية الأولى هي معرفة الوجهة.

لنسخ عدة ملفات إلى دليل موجود، يأتي اسم الدليل في النهاية:

```
cp file1 file2 dir
```

الصيغة `cp file1 file2` تعني شيئًا مختلفًا بحسب الوجهة: إن كانت `file2` ملفًا فهي اسم النسخة، وإن كانت دليلًا فستكون النسخة داخله. ولا يعني تشابه الاسم أن الأمر سينشئ دليلًا مفقودًا من تلقاء نفسه.

2.3.3 الأمر `mv`

يستخدم `mv` لنقل الملفات أو إعادة تسميتها. جرّب مع النسخة التي أنشأتها، لا مع الملف الأصلي:

```
mv copy.txt renamed.txt
mv renamed.txt dir
```

الأمر الأول يغيّر الاسم داخل الدليل نفسه، والثاني ينقل الملف إلى `dir`. لا تبقى نسخة مستقلة في المكان القديم كما يحدث مع `cp`. وقد يستبدل `mv` وجهة موجودة؛ لذلك تحقق من الاسمين، واستخدم `-i` حين تريد تأكيد الاستبدال.

في صيغة نقل عدة ملفات، تأتي الوجهة الدليلية أخيرًا أيضًا. وعلى مستوى التنفيذ، قد يكون النقل بين نظامي ملفات مختلفين أعقد من مجرد تبديل اسم، لكنك لا تحتاج إلى تلك التفاصيل كي تفهم الفرق العملي بين النقل والنسخ.

2.3.4 الأمر `touch`

ينشئ `touch` ملفًا فارغًا إن لم يكن الاسم موجودًا:

```
touch empty.txt
ls -l empty.txt
```

أما إذا كان الملف موجودًا، فلا يمحو محتواه؛ بل يحدّث أوقاته، ومنها وقت التعديل والوصول في الاستخدام الافتراضي. لهذا ليس دقيقًا وصفه بأنه «أمر إنشاء ملفات» فقط. لا تستخدمه لإثبات أن ملفًا جديد تمامًا: افحص ما إذا كان الاسم موجودًا من قبل.

2.3.5 الأمر `rm`

يحذف `rm` اسم ملف. لا ينقله عادة إلى سلة محذوفات توفر تراجعًا مضمونًا. ابدأ بملف التجربة الفارغ، واطلب التأكيد:

```
rm -i empty.txt
```

اقرأ الاسم في السؤال قبل الموافقة. الصيغة الأساسية التي يعرضها الأصل هي `rm file`، لكننا لا ننصح بتجربة الحذف على بيانات مهمة. ولا يعني حذف الاسم محوًا آمنًا لمحتوى وسيط التخزين؛ استرجاع البيانات أو التخلص الآمن منها موضوع مختلف.

تنبيه: إذا استعملت متغيرًا أو نمطًا بدل اسم صريح، فقد يستهدف الأمر ملفات أكثر مما تتوقع. افحص القيم والتوسعات أولًا، ولا تستخدم صلاحيات المدير لتجاوز مقاومة عملية حذف لا تفهم سببها.

2.3.6 الأمر `echo`

يكتب `echo` وسائطه إلى الإخراج القياسي، ويضع عادة مسافة بينها ثم نهاية سطر:

```
echo Hello again.
```

المخرجات:

```
Hello again.
```

سنستخدمه لرؤية قيم متغيرات وتوسعات أسماء. لكن `echo` ليس عارضًا دقيقًا لحدود الوسائط أو لجميع المحارف الخاصة؛ تختلف بعض تفاصيله بين الصدف والتنفيذات.

إضافة تعليمية: حين يهمك ضبط التنسيق، استخدم `printf` بصيغة واضحة، مثل `printf '%s\n' "$value"`. تعني `%s` طباعة قيمة نصية، وتعني `\n` نهاية سطر. اجعل صيغة التنسيق ثابتة، ومرّر البيانات كوسائط بعدها، بدل وضع نص غير موثوق في موضع الصيغة.

2.4 التنقل بين الأدلة

تبدأ شجرة الملفات من **الدليل الجذر** `/`. الفاصل بين مكونات المسار هو الشرطة المائلة `/`، لا الشرطة العكسية `\`. لا تخلط بين `/` وحساب المستخدم `root` أو دليل منزله الذي يكون عادة `/root`.

يسمى المسار الذي يبدأ بـ/ **مسارًا مطلقًا**، مثل `/usr/lib`. أما `dir/file1` فمسار نسبي يفسر انطلاقًا من دليل العمل الحالي للعملية. يمثل . الدليل الحالي، ويمثل .. الدليل الأب.

إذا كنت داخل `/usr/lib`، فسيشير .. إلى `/usr`، و`../bin` إلى `/usr/bin`، و. إلى `/usr/lib`. ويمكن أن يشير `/X11` إلى الموضع نفسه الذي يشير إليه `X11` هناك. وجود هذا المسار مثال توضيحي؛ ليس مطلوبًا أن يحتوي جهازك على دليل بهذا الاسم.

2.4.1 الأمر `cd`

يغيّر `cd` دليل العمل الحالي **للصدفة نفسها**:

```
cd dir
pwd
cd ..
```

إذا كنت بدأت في دليل التجربة، تدخل إلى دليله الفرعي ثم تعود إليه. أما `cd` دون وسيط فيعيدك عادة إلى دليل منزلك. لا تستخدمه ثم تواصل أمثلة الحذف معتقدًا أنك ما زلت داخل المختبر.

إضافة تعليمية: `cd` أمر مدمج في الصدفة، لأن برنامجًا ابنًا لا يستطيع أن يغيّر دليل عمل الصدفة الأم بمجرد تغيير دليله هو. هذا تطبيق صغير للعلاقة بين العمليات التي شرحناها في الفصل الأول.

2.4.2 الأمر `mkdir`

ينشئ `mkdir` دليلًا جديدًا:

```
mkdir empty-dir
```

إذا كان الاسم موجودًا، أو كانت بعض الأدلة الأب غير موجودة، فقد يفشل الأمر. يتيح `mkdir -p` إنشاء سلسلة الأدلة الأب المطلوبة، لكن افحص المسار قبل استخدامه؛ عدم ظهور خطأ ليس برهانًا على أنك كتبت الوجهة المقصودة.

2.4.3 الأمر `rmdir`

يحذف `rmdir` دليلًا فارغًا فقط:

```
rmdir empty-dir
```

رفض حذف دليل غير فارغ سلوك وقائي مفيد، لا عطل يجب تجاوزه دائمًا. يعرض الأصل أيضًا خيارَي `rm -r` للحذف التكراري و`-f` لعدم طلب التأكيد والتعامل مع أسماء مفقودة. **الحذف بواسطة `rm -rf` قد يمحو شجرة كاملة، ولا نجعله تمرينًا في هذا الفصل.** لا تجمعه مع `*` أو مسار لم تتحقق منه، ولا تخلط بين اسم دليل التجربة والدليل الجذر.

2.4.4 توسعة أنماط أسماء الملفات

تستطيع الصدفة مطابقة أنماط مع الأسماء، وتسمى العملية **توسعة أسماء الملفات** أو globbing. أهم رمزين في البداية هما `*`، الذي يطابق صفرًا أو أكثر من المحارف، و`?`، الذي يطابق محرفًا واحدًا.

```
echo *
echo '*'
```

توسع الصدفة النجمة غير المقتبسة في الأمر الأول إلى أسماء مطابقة، **قبل** تشغيل `echo`. أما الاقتباس المفرد في الثاني فيمنع التوسعة، فيطبع نجمة حرفية. الأداة لا ترى عادة النمط بعد التوسعة؛ ترى قائمة الأسماء الناتجة.

النمط	ما يطابقه عادة
`at*`	أسماء تبدأ بـ`at`
`*at`	أسماء تنتهي بـ`at`
`*at*`	أسماء تتضمن `at`
`b?at`	أسماء مثل `boat` و`brat`
`*.*`	أسماء تحتوي على نقطة، وليس جميع الأسماء

لا تحتاج أسماء ملفات يونكس إلى امتداد. والنجمة العادية لا تشمل افتراضيًا الأسماء التي تبدأ بنقطة، كما أنها لا تعبر `/` لتبحث في شجرة كاملة.

> **توضيح حديث:** في Bash بإعداداته الافتراضية المعتادة، يبقى نمط لا يطابق أي اسم كما هو. لكن `nullglob` قد يحذفه، و`failglob` قد يجعل التوسعة خطأ، وقد تختلف صدف أخرى. إذن عبارة «النمط غير المطابق يبقى حرفيًا دائمًا» ليست قاعدة عامة. راجع دليل توسعة الأسماء، وافهم إعداد صدفتك قبل استخدام نمط مع أمر يغيّر ملفات.

2.5 أوامر متوسطة المستوى

الأدوات التالية تساعدك على البحث والقراءة والمقارنة بدل فتح كل ملف يدويًا. فكر في السؤال أولًا: أأبحث عن اسم ملف أم عن سطر داخله؟ أريد كل الناتج أم بدايته؟ هذا يحدد الأداة المناسبة.

2.5.1 الأمر `grep`

يبحث `grep` عن أسطر توافق نمطًا، ثم يعرض الأسطر المطابقة. مثلًا:

```
grep root /etc/passwd
grep -n alpha file1
```

يعرض الأول أسطرًا تحتوي على `root`، لا بالضرورة السطر الخاص بذلك الحساب وحده. ويضيف `n-` في الثاني رقم السطر لتسهيل الوصول إليه. عندما تبحث في عدة ملفات، تعرض الأداة عادة اسم كل ملف مع السطر المطابق.

الخيار `i-` يتجاهل حالة الأحرف في سياقاتها المناسبة، و`v-` يعكس الاختيار فيعرض الأسطر غير المطابقة. وتبحث `F-` عن نصوص ثابتة بدل التعبيرات النمطية، أما `E-` فتستخدم التعبيرات النمطية الموسعة. يعرض الأصل الاسم `egrep`؛ نستخدم هنا `grep -E` بوصفه الصيغة الواضحة التي يشرحها دليل GNU Grep.

التعبير النمطي ليس نمط اسم ملف. في التعبيرات المعتادة لـ`grep`، تطابق `.` محرفًا واحدًا ضمن السطر، وتكرر `*` العنصر السابق صفرًا أو أكثر من المرات، فيعني `.*` تتابعًا من المحارف. أما `*` وحدها في توسعة الصدفة فلها معنى آخر.

```
grep '^root:' /etc/passwd
grep -E 'alpha|gamma' file1 file2
```

يقيد `^` المطابقة ببداية السطر في الأول. ونضع التعبير بين اقتباسين كي لا تعالج الصدفة رموزه قبل أن يراه `grep`. يقارن الأمر الثاني بين بديلين يفصل بينهما رمز الأنبوب داخل التعبير؛ ليس أنبوبًا بين برنامجين هنا لأنه مقتبس.

البحث مثل `grep root /etc/*` لا ينزل تلقائيًا إلى الأدلة الفرعية، وقد يطبع رسائل عن أدلة أو ملفات غير مقروءة. هذا لا يبرر إضافة `sudo` أو إخفاء الأخطاء قبل فهمها. **إضافة تعليمية:** حالة خروج `grep` تكون عادة صفرًا عند وجود مطابقة، وواحدًا عند عدم وجودها، وقيمة أخرى عند الخطأ؛ غياب المطابقة ليس بالضرورة عطلًا.

2.5.2 الأمر `less`

يعرض `less` النص صفحة طرفية بعد أخرى، دون أن يكون محررًا له. جرّب ملفًا تستطيع قراءته:

```
less /etc/passwd
```

المسافة تتقدم شاشة، و`b` يعود شاشة، و`q` يخرج. اكتب `/word` ثم Enter للبحث إلى الأمام، أو `?word` إلى الخلف، واضغط `n` لمتابعة البحث. البحث هنا داخل النص المعروض، لا داخل أسماء ملفات جهازك.

يمكن استقبال خرج أمر آخر:

```
grep -n root /etc/passwd | less
```

سنشرح الأنبوب `|` في القسم 2.14. يستخدم الأصل ملف القاموس `/usr/share/dict/words` مثالًا لنص كبير، قد لا يكون مثبتًا عندك، لذلك لا تجعل غيابه عقبة. بعض الأنظمة المصغرة توفر `more` دون `less`، مع اختلاف بعض المفاتيح والقدرات.

2.5.3 الأمر `pwd`

يعرض `pwd` دليل العمل الحالي. لا تفترض أن المحث يعرضه، فقد يكون مختصرًا أو مخفيًا. جرّب:

```
pwd
pwd -P
```

تعرض الصدفة عادة مسارًا منطقيًا يحتفظ بأسماء الروابط الرمزية التي استخدمتها للتنقل. يطلب `-P` المسار الفيزيائي بعد حل الروابط. يمكن أن تصل إلى الموضع نفسه عبر اسمين مختلفين؛ سنرى سبب ذلك في القسم 2.17.2.

2.5.4 الأمر `diff`

يقارن `diff` ملفين نصيين ويصف الفروق بينهما:

```
diff file1 file2
diff -u file1 file2
```

تعرض u- فرقًا موحدًا مناسبًا للمراجعة وتبادل الرقع. ترى عادة سطرَي اسمَي الملفين، ثم مقاطع سياق، وعلامة - قبل سطر من النسخة القديمة و+ قبل سطر من الجديدة. ليست هذه العلامات أوامر لإدخالها في الصدفة.

إضافة تعليمية: إذا تساوى الملفان فلا تظهر فروق عادة، وتكون حالة الخروج صفرًا. وإذا اختلفا فالحالة 1 نتيجة متوقعة للمقارنة، وليست دليلًا على فشل الأداة. حالات الخطأ الفعلي تُميَّز عن ذلك. لا تستبدل إحدى النسختين قبل قراءة الفروق.

2.5.5 الأمر `file`

إذا لم تعرف صيغة ملف، اطلب تقديرًا من `file`:

```
file file1
```

تفحص الأداة مؤشرات من المحتوى، ولا تعتمد على الامتداد وحده. قد يكون اسم ينتهي بـ`txt`. لملف غير نصي. لكن التقدير ليس ضمانًا لسلامة الملف، ولا موافقة على تشغيل ملف تنفيذي مجهول. وإذا لم تكن الأداة مثبتة في نظام مصغر، فهذه مسألة توفر حزمة لا مشكلة في الملف نفسه.

2.5.6 الأمران `find` و`locate`

يبحث `find` في شجرة فعلية تبدأ من مسار تختاره. مثال داخل دليل التجربة:

```
find . -name 'file*' -print
```

تعني . نقطة البدء، و`-name` اختبار الاسم، و`-print` طباعة المطابقات. الاقتباس حول `file*` مهم: نريد أن يرى `find` النمط ويطبقه في الأدلة التي يزورها، لا أن تستبدله الصدفة بأسماء من الدليل الحالي أولًا.

توجد خيارات تنفذ أوامر أو تحذف نتائج البحث، لكن **لا تبدأ بـ**`-exec` **أو** `-delete`. أتقن البحث والعرض أولًا، وتحقق من مجموعة النتائج قبل التفكير في تغييرها.

أما `locate` فيبحث عادة في قاعدة بيانات أُنشئت مسبقًا، ولذلك قد يكون أسرع، لكنه قد يفوّت ملفًا جديدًا أو يعرض اسمًا حُذف منذ آخر تحديث. تختلف التنفيذات وسياسات الفهرسة والوصول، وقد لا يكون مثبتًا. لا تستنتج من عدم وجود نتيجة أن الملف غير موجود قطعًا.

2.5.7 الأمران `head` و`tail`

يعرض `head` بداية ملف أو تدفق، و`tail` نهايته. الافتراضي المعتاد عشرة أسطر، ويمكن تحديد العدد بوضوح:

```
head -n 5 /etc/passwd
tail -n 5 /etc/passwd
tail -n +3 file1
```

توضيح صياغي: نستخدم `-n 5` و`-n +3` بدل بعض الصيغ التاريخية المختصرة في الأصل، كي يكون المعنى ظاهراً وأقرب إلى التوثيق الحالي.

الخيار `tail -f` مفيد لمتابعة إضافات إلى ملف سجل، لكنه لا ينتهي فورًا؛ أوقف المتابعة بـCtrl-C عندما تنتهي. وهذا يختلف عن قراءة آخر أسطر الملف مرة واحدة.

2.5.8 الأمر `sort`

يرتب `sort` أسطر النص ويكتب النتيجة إلى خرجه. افتراضيًا لا يغيّر ملف الإدخال. يطلب `-n` ترتيبًا عدديًا، و `-r` يعكس الترتيب.

```
printf '%s\n' 10 2 1 | sort -n
```

الناتج لهذا المثال العددي:

```
1
2
10
```

قد يعتمد ترتيب النصوص على الإعداد المحلي `locale`، وليس على ترتيب بايتات ثابت في كل نظام. نستخدم `LC_ALL=C` في بعض المختبرات للحصول على نتائج قابلة للمقارنة مع بيانات ASCII. ولا تكتب `sort file1 > file1`: فقد تفرغ الصدفة الملف قبل أن يقرأه البرنامج. سيصبح سبب ذلك واضحًا في قسم إعادة التوجيه.

2.6 تغيير كلمة المرور وصدفة الدخول

يغيّر `passwd` كلمة مرور حسابك عندما تسمح سياسة النظام بذلك. يطلب عادة كلمة المرور الحالية، ثم الجديدة مرتين، ولا يعرض المحارف أثناء كتابتها. **هذا إجراء فعلي على الحساب، لا تمرين مطلوب في المختبر.** لا تضع كلمة المرور في سطر أمر أو ملف أمثلة أو رسالة مراجعة.

> **توضيح حديث:** ينصح الأصل باختصار جملة إلى أحرف وتجنب جمع كلمات. لا نحول هذه النصيحة القديمة إلى قاعدة أمنية. الأهم طول مناسب، وعشوائية، وعدم إعادة الاستخدام، وتجنب القيم الشائعة أو المسرّبة؛ يمكن استخدام مدير كلمات مرور موثوق أو عبارة مرور طويلة من كلمات اختيرت عشوائيًا. ليست إضافة رقم إلى اقتباس مشهور ضمانًا للقوة. يناقش NIST كلمات المرور والعبارات الطويلة، ولا يفرض قواعد تركيب شكلية من نوع «حرف كبير ورمز» بوصفها بديلًا عن القوة الفعلية. اتبع أيضًا سياسة الحساب الذي تستخدمه.

يتيح `chsh` تغيير صدفة تسجيل الدخول إلى خيار تسمح به إعدادات النظام، وقد يستلزم المصادقة. افحص `/etc/shells` إن كان متاحًا لمعرفة الصدف المعروفة للنظام، ولا تختَر مسارًا غير صالح فتعيق تسجيل دخولك. التغيير لا يستبدل الصدفة الجارية فورًا عادة؛ ولهذا يكفي تشغيل `bash` مباشرة لتجربة هذا الفصل.

2.7 الملفات النقطية

تبدأ أسماء مثل `.bashrc` و`.login` و`.ssh` بنقطة. تسمى ملفات أو أدلة نقطية، وتستخدمها برامج كثيرة للإعداد. تخفيها بعض أدوات العرض افتراضيًا لتقليل ازدحام القائمة، لكن الإخفاء **ليس تشفيرًا ولا إذن وصول.**

للمقارنة داخل دليل التجربة:

```
touch .demo-config
ls
ls -a
ls -A
```

تظهر `-a` الأسماء النقطية وكذلك `.` و`..`، بينما تستبعد `-A` هذين الاسمين الخاصين. النقطة في بداية الاسم هي المهمة هنا؛ لا يصبح `notes.txt` ملفًا مخفيًا لأن اسمه يتضمن نقطة في الوسط.

> **توضيح حديث:** يحذر الأصل من أن النمط `*` قد يشمل `.` و`..`، ويقترح أنماطًا بديلة. في إصدارات Bash الحديثة يتحكم أيضًا الخيار `globskipdots` في ذلك، إلى جانب إعدادات مثل `dotglob`. لا تفترض سلوكًا واحدًا لكل الصدف والإصدارات، ولا تستخدم نمطًا عامًا لحذف «كل الملفات المخفية». للعرض يكفي `ls -A`؛ وللتعديل حدد أسماء تعلم ما تفعله.

2.8 متغيرات الصدفة ومتغيرات البيئة

تخزن الصدفة قيمًا نصية في **متغيرات**، بعضها تختاره أنت، وبعضها يؤثر في سلوك الصدفة. مثلًا، تتحكم `PS1` في المحث التفاعلي في Bash. للإسناد نكتب اسمًا، ثم `=`، ثم قيمة، **دون مسافات حول علامة المساواة**:

```
STUFF=blah
printf '%s\n' "$STUFF"
```

اسم المتغير هو `STUFF` دون `$`. نستخدم `$STUFF` عند طلب قيمته. إذا كتبت `STUFF = blah` فستحاول الصدفة تفسير السطر بطريقة أخرى؛ المسافات جزء من القواعد وليست زينة اختيارية.

لمتغيرات **البيئة** دور إضافي: تمرر إلى البرامج الابنة التي تبدأها الصدفة. يجعل `export` متغيرًا متاحًا في البيئة:

```
STUFF=blah
export STUFF
sh -c 'printf "%s\n" "$STUFF"'
```

يقرأ البرنامج الابن القيمة المصدَّرة. أما متغير صدفة غير مصدَّر فلا يُمرر بهذه الطريقة عند تشغيل برنامج خارجي. وحصول الابن على نسخة من البيئة لا يتيح له تعديل بيئة الصدفة الأم؛ لا تتوقع أن يغير برنامج إعدادات جلستك بعد خروجه تلقائيًا.

يقرأ `less` مثلًا المتغير `LESS` لخيارات افتراضية، وقد تجد في توثيق برنامج قسمًا باسم `ENVIRONMENT` يشرح المتغيرات التي يقرأها. تنتهي تغييرات الجلسة عادة بانتهائها ما لم تحفظ في إعداد بدء مناسب؛ سنفصل ملفات البدء في الفصل الثالث عشر.

إضافة تعليمية: الاقتباس المزدوج في `"$STUFF"` يسمح بتوسعة المتغير، لكنه يحافظ على القيمة وسيطًا واحدًا ولو احتوت مسافات. الاقتباس المفرد في `'$STUFF'` يمنع التوسعة ويعطي نصًا حرفيًا. و`unset STUFF` يزيل المتغير. لا تستخدم متغيرات البيئة خزنة للأسرار؛ قد تُورث أو تُعرض بوسائل الفحص والتسجيل.

2.9 مسار البحث عن الأوامر

يحمل المتغير PATH قائمة أدلة تفصل بينها النقطتان :. عندما تكتب اسم برنامج خارجي دون /، تستخدم الصدفة هذا المسار للبحث عنه، بعد مراعاة آلياتها الأخرى مثل الأوامر المدمجة والدوال والأسماء المستعارة.

```
printf '%s\n' "$PATH"
command -v ls
type cd
```

تعرض الأوامر المسار، وطريقة الوصول إلى ls، وطبيعة cd. ومن أمثلة قيمة PATH، لا القيمة المطلوبة على جهازك:

```
/usr/local/bin:/usr/bin:/bin
```

إذا وجد برنامج بالاسم نفسه في أكثر من موضع، فإن ترتيب البحث مهم. لإضافة دليل نثق به دون إضاعة القيمة السابقة، يمكن استخدام إسناد مثل الآتي **بعد التحقق من محتوى الدليل**:

```
PATH="$HOME/bin:$PATH"
PATH="$PATH:$HOME/bin"
```

الصيغة الأولى تضعه في البداية فتمنحه أولوية، والثانية تضعه في النهاية. لا تنفذ السطرين لمجرد أنهما متجاوران؛ إنهما خياران لتوضيح الفرق.

> **إضافة تعليمية مهمة:** لا تضع . أو دليلًا يكتب فيه أشخاص غير موثوقين في بداية PATH. وقد تمثل خانة فارغة، مثل نقطتين متتاليتين أو نقطتين في طرف القيمة، الدليل الحالي أيضًا. افحص ما ستشغله، ولا تمنح برنامجًا مجهولًا اسمًا يحجب أداة نظام معروفة.

الأمر الذي يحتوي على /، مثل ./demo.sh أو /usr/bin/ls، يحدد مسارًا ولا يحتاج إلى بحث الأسماء المعتاد في PATH. أما كتابة demo.sh وحده فلا تضمن تشغيل ملف بهذا الاسم من الدليل الحالي.

إذا أتلفت PATH في الجلسة، لا تبدأ برنامجًا ابنًا منها متوقعًا أن يصلحه؛ قد يرث القيمة التالفة. يمكنك إعادته إلى قيمة حفظتها سابقًا، أو فتح جلسة جديدة من بيئة سليمة. وإذا كان الخطأ في ملف بدء، فقد تتكرر المشكلة حتى تصلح ذلك الملف. لهذا نجرب التغييرات داخل جلسة محدودة، ونقرأ القيمة قبل تعديلها.

2.10 المحارف الخاصة

للمحرف الواحد معانٍ متعددة بحسب السياق: النجمة في نمط اسم ليست كنجمة التعبير النمطي، والأنبوب داخل نص مقتبس ليس ربطًا بين عمليتين. يعرض الجدول الأسماء والأدوار للتعرف إليها، لا قاموسًا يجعل كل رمز ذا معنى ثابت.

الجدول 2-1 — محارف شائعة وأدوارها.

المحرف	الاسم	أمثلة على الاستخدام
*	نجمة، Asterisk	توسعة الأسماء؛ تكرار عنصر سابق في تعبير نمطي
.	نقطة، Dot	الدليل الحالي؛ جزء من اسم؛ محرف عام في تعبير نمطي
!	علامة تعجب، Bang	نفي أو توسعة سجل الأوامر في سياقات معينة
|	أنبوب، Pipe	وصل خرج أمر بإدخال آخر؛ أو اختيار بديل داخل تعبير مناسب
/	شرطة مائلة، Slash	فصل مكونات المسار؛ بدء البحث في عارض مثل `less`
\	شرطة عكسية، Backslash	حماية محرف أو متابعة سطر؛ ليست فاصل الأدلة هنا
$	علامة الدولار، Dollar	توسعة متغير؛ نهاية السطر في تعبير نمطي
'	اقتباس مفرد، Single quote	إبقاء النص حرفيًا داخل الاقتباس
`	اقتباس خلفي، Backtick	صيغة قديمة لاستبدال الأوامر؛ الصيغة `$(...)` أوضح عادة
"	اقتباس مزدوج، Double quote	حماية تجميع النص مع السماح ببعض التوسعات
^	علامة القبعة، Caret	بداية سطر أو نفي داخل بعض أنماط المطابقة
~	تلدة، Tilde	اختصار للدليل المنزلي في الموضع المناسب من كلمة الصدفة
#	علامة العدد، Hash	تعليق في الصدفة عند بدء كلمة غير مقتبسة؛ وأدوار أخرى في لغات أخرى
[ ]	قوسان مربعان، Brackets	مجموعات محارف أو نطاقات في أنماط معينة
{ }	قوسان معقوفان، Braces	تجميع أو توسعات في الصدف واللغات التي تدعمها
_	شرطة سفلية، Underscore	جزء من اسم، ويستخدم أحيانًا بدل المسافة

تكتب وثائق الطرفية اختصارات تحكم مثل ^C للدلالة على Ctrl-C؛ لا يعني ذلك إدخال حرفَي ^ وC. **إضافة تعليمية:** تسمح مسافات في أسماء الملفات على لينكس، لكن عليك اقتباس الاسم، مثل `cat 'my notes.txt'`. تغيير المسافة إلى _ اختيار للتسمية، لا شرط يفرضه النظام.

2.11 تحرير سطر الأوامر

لا تضطر إلى إعادة كتابة سطر كامل لأن حرفًا أخطأ. في Bash التفاعلية يمكنك التحرك بالأسهم، واسترجاع أوامر سابقة، وتعديل السطر قبل الضغط على Enter. استرجاع أمر لا يشغله حتى تؤكد التنفيذ؛ اقرأ السطر المسترجع، وخصوصًا أوامر الحذف أو الصلاحيات.

الجدول 2-2 — مفاتيح شائعة في نمط التحرير المعتاد الشبيه بـEmacs.

المفتاح	الوظيفة
Ctrl-B	التحرك إلى الخلف محرفًا في السطر
Ctrl-F	التحرك إلى الأمام محرفًا
Ctrl-P	استرجاع الأمر السابق من السجل
Ctrl-N	الانتقال إلى الأمر التالي في السجل
Ctrl-A	الانتقال إلى بداية السطر
Ctrl-E	الانتقال إلى نهاية السطر
Ctrl-W	حذف الكلمة السابقة في الإعداد المعتاد
Ctrl-U	حذف الجزء من المؤشر إلى بداية السطر
Ctrl-K	حذف الجزء من المؤشر إلى النهاية
Ctrl-Y	إعادة إدراج نص حُذف بأوامر التحرير المناسبة

إضافة تعليمية: يكمل Tab اسمًا أو يعرض إمكانات الإكمال في إعدادات شائعة. ويبدأ Ctrl-R بحثًا عكسيًا في السجل، بينما يعيد Ctrl-L عرض الشاشة في كثير من التطبيقات. لا تستخدم Ctrl-R بوصفه أمر تنظيف عام للشاشة. يمكن تغيير ربط المفاتيح أو اختيار نمط تحرير شبيه بـvi؛ عندها قد تختلف بعض الوظائف.

التاريخ التفاعلي أداة مساعدة لا سجل تدقيق أمني كامل. وقد تُحفظ الأوامر التي تدخلها فيه؛ لا تكتب كلمات المرور أو الرموز السرية ضمن أمر تتوقع أن يبقى في الذاكرة فقط.

2.12 محررات النصوص

تُحفظ إعدادات كثيرة في ملفات نصية، لذلك تحتاج إلى محرر يحافظ على النص بدل إضافة تنسيق معالج كلمات. لا توجد فائدة من معرفة اسم كل أداة إذا كنت لا تستطيع تعديل سطر دون إفساد الملف.

يقدم الأصل `vi` و`Emacs` بوصفهما أداتين أساسيتين في ثقافة يونكس. يستخدم `vi` أنماطًا مختلفة للإدخال والأوامر، وتوجد منه نسخ مثل Vim. أما Emacs فبيئة تحرير واسعة تحتوي على مساعدة وتوسعات، وقد تعمل بواجهة رسومية أو داخل الطرفية.

إضافة تعليمية: لا نلزم المبتدئ باختيار أداة معقدة من اليوم الأول. يمكنك البدء بمحرر نصي بسيط مثل `nano` إذا كان مثبتًا، ثم تعلم الأداة التي تناسبك. المهم أن تعرف فتح ملف وحفظه والخروج دون حفظ، وأن تتدرب على ملف تجريبي لا ملف نظام.

- في Vim/`vi`، اضغط `i` للدخول إلى وضع الإدخال، و`Esc` للعودة إلى وضع الأوامر. يحفظ `:w` الملف، ويخرج `:q` إذا لم توجد تغييرات معلقة، ويحفظ `:wq` ثم يخرج. أما `:q!` فيتخلى عن التغييرات غير المحفوظة؛ اقرأ المقصود قبل استعماله.
- في Emacs، يبدأ `Ctrl-H t` الدرس المدمج المعتاد. يعني `Ctrl-X Ctrl-S` حفظ الملف، و`Ctrl-X Ctrl-C` الخروج، مع الأسئلة اللازمة للتغييرات غير المحفوظة.
- في nano، تعرض الشاشة مفاتيح الحفظ والخروج، مثل `Ctrl-O` و`Ctrl-X` في الإعداد المعتاد. قراءة الشريط السفلي أفضل من التخمين.

خذ نسخة احتياطية قبل تعديل إعداد مهم، ولا تخلط بين ملف تملكه ورابط رمزي يقود إلى ملف آخر. تشغيل محرر بصلاحيات واسعة ليس وسيلة اعتيادية لتعلم التحرير، وبعض ملفات النظام تحتاج أداة متخصصة تفحص بنيتها، كما سنرى مع `visudo`.

2.13 الحصول على المساعدة والتوثيق

المساعدة «المتاحة على النظام» لا تعني دائمًا اتصالًا بالإنترنت. توفر صفحات الدليل المحلية وصفًا للأمر وخياراته ومخرجاته وإحالاته. مثلًا:

```
man ls
man -k sort
man 5 passwd
```

يعرض الأول صفحة `ls`، ويبحث الثاني في أوصاف الصفحات عن كلمة `sort`، ويطلب الثالث قسمًا معينًا. قد لا تكون حزم الدليل أو فهارسه مثبتة في بيئة مصغرة، فلا تفسر غياب صفحة على أنه غياب للأداة نفسها.

ليست صفحة الدليل دائمًا درسًا للمبتدئ؛ قد تسرد عشرات الخيارات بالترتيب الأبجدي. ابدأ بالوصف العام والصيغة، ثم ابحث داخل الصفحة عن الخيار المطلوب. إذا استُخدم `less` للعرض، تنفع مفاتيح `/` للبحث و`q` للخروج التي تعلمتها.

الجدول 3-2 — أقسام شائعة في دليل يونكس.

القسم	المحتوى المعتاد
1	أوامر المستخدم
2	نداءات النظام
3	دوال المكتبات وواجهاتها البرمجية
4	واجهات الأجهزة وملفاتها وبرامج تشغيلها
5	صيغ الملفات وملفات الإعداد
6	الألعاب
7	موضوعات عامة واتفاقيات ومعايير ونظرات شاملة
8	أوامر إدارة النظام وخدماته

لهذا تختلف `passwd(1)` عن `passwd(5)`: الأولى تتعلق بأمر تغيير كلمة المرور، والثانية بصيغة ملف الحسابات. الرقم ليس رقم إصدار، ولا جزءًا من اسم البرنامج الذي تنفذه.

للمساعدة السريعة تدعم أدوات GNU كثيرة `--help`، لكن `-h` **ليس اختصارًا مضمونًا للمساعدة**؛ قد يعني إخراجًا بوحدات مقروءة مثلًا. وتوفر `info` توثيقًا أكثر تفصيلًا لبعض أدوات GNU:

```
ls --help
info coreutils
```

تفحص أيضًا `/usr/share/doc/` لملاحظات الحزم وأمثلتها، ووثائق التوزيعة أو المشروع على الويب. عند نسخ حل من الإنترنت، تحقق من إصدار الأداة ومن أثر الأمر. المعلومات الكثيرة لا تعفيك من التمييز بين وصف وظيفة وأمر يغيّر النظام.

2.14 إدخال الصدفة وإخراجها

يمكن تحويل وجهة الإخراج القياسي من الطرفية إلى ملف باستخدام >، أو إضافة الناتج إلى نهاية ملف باستخدام >>. مثال على **ملف تجريبي جديد** داخل المختبر:

```
printf '%s\n' 'first line' > output.txt
printf '%s\n' 'second line' >> output.txt
cat output.txt
```

المخرجات بعد السطرين:

```
first line
second line
```

تفتح الصدفة وجهة > للكتابة، وقد تنشئها أو **تفرغ محتواها السابق قبل تشغيل الأمر**. حتى فشل الأمر بعد ذلك لا يعيد المحتوى. لهذا كان `sort file1 > file1` خطأً، ولهذا ينبغي التأكد من الاسم الذي يلي > مثلما نتأكد من اسم تحذفه.

يمكن تفعيل `noclobber` في Bash بواسطة `set -C` للحد من استبدال ملفات عادية موجودة بإعادة التوجيه. هذا حاجز إضافي له حدود، لا نسخ احتياطي ولا حماية عامة من كل برنامج يكتب ملفًا. أعده بإيقاف الخيار بواسطة `set +C` إذا كنت فعلته للتجربة فقط.

يربط الأنبوب | خرج أمر بإدخال أمر آخر. مثلًا:

```
printf '%s\n' alpha beta | tr 'a-z' 'A-Z'
```

يحوّل `tr` الأحرف اللاتينية الصغيرة في هذا المثال إلى كبيرة. نقتبس النطاقين كي لا تعالجهما الصدفة كنمط أسماء في سياق آخر. يعرض الأصل الفكرة مع `head /proc/cpuinfo`؛ نستخدم بيانات صغيرة ثابتة بدل افتراض بنية معلومات معالج بعينه.

يمكن وصل أكثر من أداة، كأن تصفّي بـ`grep` ثم ترتب بـ`sort` ثم تعرض بـ`less`. لا يعني الأنبوب إنشاء ملف وسيط دائم، كما أنه لا يرسل `stderr` إلى الأمر التالي تلقائيًا.

2.14.1 الإخراج القياسي للأخطاء

للعمليات ثلاث واصفات ملفات قياسية نستخدمها كثيرًا:

الرقم	الاسم	الغرض المعتاد
0	stdin	بيانات الإدخال
1	stdout	النتائج العادية
2	stderr	رسائل التشخيص والأخطاء

مثال فشل مقصود: داخل دليل تجربة لا يحتوي على `missing-demo-file`، شغّل:

```
ls missing-demo-file > listing.txt
```

يظل الخطأ ظاهرًا في الطرفية لأنك حوّلت القناة 1 فقط، بينما قد يكون `listing.txt` قد أُنشئ فارغًا. لتسجيل كل قناة منفصلة:

```
ls missing-demo-file > listing.txt 2> errors.txt
```

يحدد `2>` رقم القناة مباشرة؛ لا تضع مسافة بين 2 و> فتغيّر معنى السطر. ويمكن جعل القناتين تتجهان إلى الملف نفسه:

```
ls missing-demo-file > combined.txt 2>&1
```

تفسر الصدفة التوجيهات من اليسار إلى اليمين. يوجّه السطر الأخير `stdout` أولًا إلى الملف، ثم يجعل `stderr` يشير إلى وجهة `stdout` الحالية. **لا يعادل ذلك** ترتيب `2>&1 > combined.txt`؛ ففيه تُنسخ وجهة الخرج القديمة للأخطاء قبل تغيير وجهة النتائج.

قد يخفي تحويل الأخطاء إلى ملف تفسير فشل أمر، فلا تخلط بين «لم أرَ رسالة» و«نجح». اقرأ ملف الأخطاء وحالة الخروج. وتذكّر أن علامة `&` في `2>&1` جزء من صيغة نسخ واصف، وليست علامة تشغيل في الخلفية.

2.14.2 إعادة توجيه الإدخال القياسي

يقرأ البرنامج من ملف بوصفه إدخاله القياسي عند استخدام <:

```
head -n 2 < file1
```

يمكن أن تحقق `head -n 2 file1` نتيجة شبيهة، لكن المسؤولية تختلف: في صيغة < تفتح الصدفة الملف وتوصله بالإدخال، وفي صيغة الوسيط يتولى البرنامج التعامل مع اسم الملف. بعض البرامج يقرأ فقط من الإدخال القياسي، فيصبح هذا الفرق عمليًا لا اصطلاحيًا فقط.

2.15 فهم رسائل الخطأ

رسالة الخطأ قرينة تشخيصية، وليست إهانة للمستخدم ولا دعوة فورية إلى رفع الصلاحيات. اقرأ اسم الأداة، والمورد أو العملية التي ذكرتها، والسبب المبلغ عنه. قد تختلف الصياغة واللغة باختلاف البرنامج والإعداد المحلي.

2.15.1 تشريح رسالة خطأ في يونكس

مثال توضيحي شائع، قد تختلف علامات الاقتباس فيه:

```
ls: cannot access 'missing-demo-file': No such file or directory
```

يتكون المعنى من ثلاث قطع: `ls` هو المبلّغ، و`missing-demo-file` هو الاسم الذي حاول الوصول إليه، و `No such file or directory` هو سبب الفشل. إذا وصلتك الرسالة من سكربت طويل، فابدأ بالسطر الذي استدعى الأداة، لا باسم السكربت وحده.

يعرض الأصل برنامجًا افتراضيًا يفشل لأن `/etc/scumd/config` مفقود ثم يطبع سلسلة أخطاء لاحقة. الفكرة أن تعالج **السبب الأول ذي الصلة** بدل مطاردة كل نتيجة لاحقة. **توضيح تعليمي:** لا تنشئ ملف إعداد فارغًا عشوائيًا لإسكات الرسالة؛ تحقق من المسار والتثبيت والتوثيق والمحتوى المطلوب، فقد يكون غياب الملف عرضًا لخطأ آخر.

التحذير `warning` قد يعني أن البرنامج سيحاول الاستمرار، لكنه لا يضمن نجاحًا كاملًا. ولا تستلزم كل رسالة تحذير قتل عملية. دوّن الأمر الذي نفذته وسياقه، وافحص حالة الخروج بدل الاكتفاء بمظهر الشاشة.

2.15.2 أخطاء شائعة

- `No such file or directory`: الاسم غير موجود، أو أحد مكونات المسار مفقود، أو رابط يقود إلى هدف غائب. افحص المسار من بدايته، وليس اسم الملف الأخير فقط.
- `File exists`: تحاول إنشاء كيان لا تسمح العملية بوجوده مسبقًا، مثل دليل باسم مستخدم لملف. لا تحذف الموجود حتى تعرف ما هو.
- `Not a directory` و`Is a directory`: استُخدم ملف كدليل أو دليل كملف في عملية لا تقبل ذلك. إذا كان `a` ملفًا عاديًا، فلا يمكنك إنشاء `a/b` تحته.
- `No space left on device`: تعذر تخصيص مورد تخزين. **إضافة تعليمية:** قد تكون المساحة ممتلئة أو نفدت عُقد `inode` في نظام ملفات، فلا تحذف ملفات مهمة عشوائيًا قبل التشخيص.
- `Permission denied`: لا تسمح الأذونات أو سياسة الوصول بالعملية. يمكن قراءة برنامج مع عدم وجود إذن لتنفيذه، وقد يكون المانع في أحد الأدلة على الطريق.
- `Operation not permitted`: العملية المطلوبة محظورة بحسب الصلاحيات أو السياسة، مثل محاولة إرسال إشارة إلى عملية لا تملك حق التأثير فيها. ليست مرادفًا لـ«الحل هو sudo».
- `Segmentation fault` و`Bus error`: حدث خطأ وصول إلى الذاكرة أو تعامل معها. قد يكون عيبًا برمجيًا أو مشكلة أعمق تحتاج تشخيصًا؛ لا تُصلح عادة بتغيير أذونات ملف أو تكرار الأمر بصلاحيات المدير.

مثال خطأ مقصود داخل دليل تجريبي جديد:

```
touch regular-demo
touch regular-demo/child
```

تفشل العملية الثانية لأن `regular-demo` ملف لا دليل. فكر في السبب قبل تغيير أي شيء: وجود اسم صحيح في المسار لا يكفي؛ يجب أن يكون نوعه مناسبًا لدوره.

إضافة تعليمية: تعرض `printf '%s\n' "$?"` حالة خروج **الأمر السابق مباشرة**. الصفر نجاح في العرف المعتاد، لكن تفسير القيم الأخرى خاص بالأداة؛ تذكر أن `diff` و`grep` يستخدمان `1` لنتيجة متوقعة في حالات معينة. وأي أمر تنفذه بعد الفشل سيغيّر القيمة التي تراها في `$?`.

2.16 عرض العمليات والتعامل معها

العملية برنامج قيد التنفيذ، ولها معرّف PID ضمن البيئة التي تراها. يوفر ps لقطة لحالة العمليات، لا متابعة حية لكل تغير. ابدأ بـps، ثم اطلب الأعمدة التي تهمك:

```
ps
ps -p "$$" -o pid,ppid,tty,stat,time,args
```

تمثل $$ في الصدفة المعتادة معرّفها. تعرض الأعمدة المطلوبة معرّف العملية وأمها والطرفية والحالة ووقت المعالج وسطر الأمر. قد تتغير قائمة العمليات وأسماؤها حتى أثناء القراءة؛ لا تحفظ رقمًا من مثال لتستعمله لاحقًا في أمر قتل.

من معاني الأعمدة الشائعة:

- TTY: الطرفية المرتبطة بالعملية إن وجدت، وقد تظهر ? عندما لا توجد طرفية متحكمة.
- STAT: رموز حالة العملية؛ مثل S للانتظار القابل للمقاطعة، وR للتنفيذ أو الاستعداد له، وT لحالة توقف. توجد رموز أخرى وتفاصيل في ps(1).
- TIME: وقت المعالج التراكمي، لا المدة منذ بدء العملية. قد تعيش عملية ساعة وهي تنتظر معظمها وتستهلك وقت معالج قليلًا.
- COMMAND أو ARGS: اسم الأمر أو وسائطه حسب صيغة العرض. لا تعتبر النص هوية أمنية مضمونة؛ يمكن أن يتغير، وقد تُختصر معلوماته أو تحتوي بيانات حساسة.

2.16.1 خيارات الأمر

يدعم ps في تنفيذ procps الشائع أساليب خيارات من تقاليد Unix وBSD وGNU. لا تضف أو تحذف الشرطة أمام مجموعة خيارات اعتباطًا؛ قد يتغير المعنى. هذه صيغ BSD التي يعرضها الأصل:

الصيغة	الفكرة
ps x	إظهار عمليات المستخدم، بما فيها التي لا ترتبط بطرفية
ps ax	عرض أوسع لعمليات البيئة، لا عمليات المستخدم وحده
ps u	صيغة عرض تتضمن معلومات موجهة للمستخدم
ps w	عرض أعرض بدل الاكتفاء بعرض الطرفية المعتاد

`ps aux`	جمع اختيار واسع للعمليات مع صيغة العرض التفصيلية

قد يفيد تكرار `w` لعرض أطول في بعض التنفيذات. ونستخدم في المختبر `p-` لاختيار معرّف محدد و`o-` لتحديد الأعمدة صراحة، كما فعلنا في الفصل الأول. ليست `ps aux` و`ps -aux` تهجئتين مضمونتين للطلب نفسه.

إضافة تعليمية: داخل حاوية أو نظام ذي قيود، لا تعني «كل العمليات» رؤية كل شيء على الجهاز المضيف. وقد تمنع السياسات بعض التفاصيل. توجد أدوات متابعة مثل `top` وأدوات ملفات مفتوحة مثل `lsof` سنعود إليها في فصل الموارد.

2.16.2 إنهاء العمليات وإرسال الإشارات

اسم `kill` قد يوحي بأنه لا يفعل إلا القتل، لكنه يطلب من النظام **إرسال إشارة**. الاستجابة تعتمد على الإشارة وعلى البرنامج والسياسات. الصيغ التالية للشرح؛ استبدل `pid` بمعرّف عملية تجريبية تملكها فقط، ولا تنفذ كلمة `pid` حرفيًا:

```
kill pid
kill -STOP pid
kill -CONT pid
kill -KILL pid
```

يرسل الأمر الأول عادة `SIGTERM`، أي طلب الإنهاء الذي يمكن لبرنامج معالجته وترتيب موارده قبل الخروج. يوقف `SIGSTOP` التنفيذ دون إنهاء العملية، ويسمح `SIGCONT` بمتابعته. تبقى العملية المتوقفة موجودة، وليست عملية انتهت ثم أُنشئت مجددًا.

أما `SIGKILL` فلا يمكن للبرنامج التقاطها أو تجاهلها لتنظيف حالته، ولذلك هي ملاذ أخير لا خيار افتراضي. الرقم `9` يمثلها على لينكس، لكن الأسماء أوضح من حفظ أرقام إشارات قد تختلف بينها الأنظمة. ولا يمكن التقاط `SIGSTOP` أو تجاهلها أيضًا.

يتسبب Ctrl-C عادة في `SIGINT` لمجموعة العمليات الأمامية في الطرفية. ليس كل إيقاف من لوحة المفاتيح إشارة `TERM`، ولا يعني اختفاء المحث أن قتل أي عملية ظاهر اسمها مناسب.

تنبيه: قد يؤدي إنهاء محرر أو قاعدة بيانات إلى فقدان عمل غير محفوظ. تحقق من المالك والمعرّف وسطر الأمر والسياق قبل إرسال إشارة، ولا تستهدف معرّفات سالبة أو مجموعات عمليات خلال تجربة لا تفهم حدودها. مختبر نهاية الفصل يستهدف فقط عملية `sleep` ينشئها بنفسه ويحفظ معرّفها مباشرة.

2.16.3 التحكم في المهام

في الصدفة التفاعلية، تسمح **إدارة المهام** بتعليق عمل أمامي وإعادته أو تشغيله في الخلفية. يرسل Ctrl-Z عادة SIGTSTP، وهي ليست SIGSTOP نفسها رغم تشابه أثر التوقف في حالات كثيرة. ويعرض `jobs` المهام التي تتعقبها الصدفة الحالية.

- `fg` يعيد مهمة إلى المقدمة لتتفاعل مع الطرفية.
- `bg` يستأنف مهمة متوقفة في الخلفية.
- صيغة مثل `fg %1` تستخدم رقم مهمة في تلك الصدفة، لا معرّف عملية PID على النظام كله.

إذا ضغطت Ctrl-Z وأنت تريد الخروج، فالبرنامج لم ينتهِ بالضرورة. قد تترك عدة برامج معلقة دون أن تشعر. افحص `jobs`، وأعد المهمة إلى المقدمة ثم أغلقها بطريقة مناسبة. لا تفترض أن التحكم في المهام مفعل بالطريقة نفسها داخل سكربت غير تفاعلي.

يمكنك استخدام طرفيات متعددة، أو تعلم أدوات جلسات مثل `screen` و`tmux` لاحقًا. لكنها لا تلغي الحاجة إلى فهم ما هو برنامج جارٍ وما هو مجرد نافذة تعرضه.

2.16.4 العمليات في الخلفية

يتيح وضع `&` في نهاية الأمر للصدفة متابعة استقبال الأوامر دون انتظار انتهاء العمل الأمامي. مثال محدود العمر:

```
sleep 5 &
background_pid=$!
ps -p "$background_pid" -o pid,stat,comm
wait "$background_pid"
```

تحفظ `$!` معرّف آخر أمر بدأ في الخلفية. ينتظر `wait` انتهاءه عند الحاجة، ولا يترك هذا المثال عملًا دائمًا. وإذا تأخرت في إدخال الأسطر، فقد يكون `sleep` انتهى قبل أن يراه `ps`؛ الصق المثال معًا إن أردت الملاحظة.

قد تكتب عملية خلفية إلى الطرفية فتختلط رسائلها بالسطر الذي تحرره، وقد تتوقف إذا حاولت قراءة الطرفية في سياق لا يسمح بذلك. إعادة توجيه القنوات إلى وجهات مناسبة تفصل البيانات، لكنها لا تجعل كل برنامج تفاعلي صالحًا للعمل بلا مستخدم.

توضيح حديث مهم: لا يضمن `&` استمرار العملية بعد الخروج أو إغلاق الطرفية، خلافًا لتعميم في الأصل. قد يصل `SIGHUP`، وقد تعيد الصدفة إرساله إلى مهامها، وقد تنهي سياسة الجلسة العمليات. توجد وسائل مثل `nohup` و`disown` ومديري الجلسات والخدمات لمتطلبات مختلفة؛ افهم الإشارات والإدخال والإخراج وسياسة النظام قبل الاعتماد على بقاء عمل طويل. هذا الفصل لا يحوّل `&` إلى خدمة دائمة.

2.17 أنماط الملفات وأذونات الوصول

تعرض بداية السطر في `ls -l` **نوع الملف وأذوناته.** انظر إلى المثال الآتي بوصفه بيانات توضيحية:

```
-rw-r--r-- 1 amina users 7041 Sep 7 10:00 endnotes.html
```

الخانة الأولى - تعني ملفًا عاديًا. وتشير `d` في هذا الموضع إلى دليل، و`l` إلى رابط رمزي. سنقابل أنواع أجهزة وواجهات أخرى في الفصل الثالث؛ لا نفترض أن كل اسم في شجرة الملفات يحتوي نصًا عاديًا.

نقرأ سلسلة النمط من اليسار إلى اليمين

`- rw- r-- r--`

المقطع بحسب ترتيبه من اليسار	معناه
الأول: -	نوع الملف: ملف عادي في هذا المثال
الثاني: rw-	أذونات المالك: قراءة وكتابة، دون تنفيذ
الثالث: r--	أذونات المجموعة: قراءة فقط
الرابع: r--	أذونات الآخرين: قراءة فقط

الشكل 1-2 — تفكيك تعليمي لسلسلة نمط الملف. المسافات أضيفت لفصل المجموعات، ولا تظهر بهذا الشكل داخل سلسلة الأذونات الأصلية.

تتكون كل مجموعة عادية من خانات القراءة `r` والكتابة `w` والتنفيذ `x`. وتعني `-` في موضع إحدى هذه الخانات أن الإذن غير مضبوط، لا أن الملف غير موجود. وتسمى هذه الخانات أحيانًا «بتات الأذونات».

ترتبط المجموعة الأولى بمالك الملف، والثانية بالمستخدمين الذين تنطبق عليهم مجموعة الملف، والثالثة بالآخرين. يمكنك رؤية عضوياتك بواسطة `id` أو `groups`. **توضيح تقني:** لا تُجمع المجموعات الثلاث ببساطة للحصول على أوسع إذن؛ في النموذج التقليدي تطبق فئة المالك إن كنت المالك، وإلا ففئة المجموعة المناسبة، وإلا فئة الآخرين. وقد تضيف قوائم التحكم في الوصول ACL وسياسات أخرى تفاصيل لا تظهر في هذا النموذج المختصر.

قد ترى `s` بدل `x` في موضع المالك، للدلالة على الجمع بين التنفيذ وsetuid، أو `S` عندما يكون البت الخاص موجودًا دون بت التنفيذ. يسمح `setuid` في ظروف مناسبة بتغيير هوية التنفيذ الفعالة إلى هوية مالك الملف. ليس كل برنامج كهذا مملوكًا لـroot، ولا تجعل وضع هذا البت تجربة للمبتدئ. من أمثلة الحاجة التاريخية إلى امتياز خاص برنامج `passwd` لتحديث بيانات مصادقة محمية؛ الملفات والآليات الفعلية تعتمد على النظام، لا على افتراض كتابة كلمة مرور في `/etc/passwd`.

2.17.1 تعديل الأذونات

يغير `chmod` أذونات ملف تملكه أو لديك حق تعديل نمطه. تحدد `u` المالك، و`g` المجموعة، و`o` الآخرين، و`a` الجميع. تضيف `+` إذنًا، وتزيله `-`، وتعين `=` المجموعة المحددة صراحة.

داخل دليل التجربة، مع ملف غير حساس:

```
touch permissions-demo
chmod u=rw,go= permissions-demo
chmod g+r permissions-demo
chmod o+r permissions-demo
chmod go-r permissions-demo
ls -l permissions-demo
```

كان يمكن جمع إضافتي القراءة في `chmod go+r permissions-demo`. لا تستخدم `o` بمعنى المالك؛ المالك هو `u`. ولا تعالج مشكلة إذن بفتح الكتابة للجميع دون فهم سببها، خصوصًا مع شجرة مشتركة أو ملفات إعداد.

يمكن تمثيل كل مجموعة برقم ثماني: القراءة `4` والكتابة `2` والتنفيذ `1`. نجمع القيم داخل المجموعة؛ فـ`6` تعني قراءة وكتابة، و`5` قراءة وتنفيذ، و`0` لا شيء من الثلاثة.

الجدول 4-2 — أنماط عادية شائعة، مع تفسيرها.

النمط	المالك	المجموعة والآخرون	استخدام نموذجي
`644`	قراءة وكتابة	قراءة	ملف يمكن للآخرين قراءته إذا سمحت بقية المسارات والسياسات
`600`	قراءة وكتابة	لا أذونات	ملف خاص

755	قراءة وكتابة وتنفيذ	قراءة وتنفيذ	دليل عام أو برنامج قابل للتنفيذ
700	قراءة وكتابة وتنفيذ	لا أذونات	دليل خاص أو برنامج خاص
711	قراءة وكتابة وتنفيذ	تنفيذ فقط	دليل يمكن عبوره دون عرض قائمة أسمائه

الصيغة `chmod 644 permissions-demo` تعيّن البتات العادية للمجموعات الثلاث. **توضيح تقني:** لا نعمم أنها تمحو كل بت خاص في كل حالة؛ يحافظ `chmod` GNU مثلًا على بعض بتات `setuid` و`setgid` للأدلة ما لم يُطلب تغييرها صراحة. لا تحتاج إلى تلك البتات لإتقان الحالات العادية في الجدول.

للأدلة معانٍ مختلفة قليلًا:

- `r` يسمح بقراءة قائمة الأسماء.
- `x` يسمح بالبحث/العبور والوصول إلى أسماء داخله وفق باقي الأذونات.
- `w` يتيح تعديل مدخلات الدليل، مثل إنشاء اسم أو حذفه، ويحتاج الاستخدام المعتاد إلى إذن العبور أيضًا.

لذلك قد تتمكن من حذف اسم ملف لا يسمح بكتابة محتواه إذا سمحت أذونات الدليل والسياسات بذلك. وبالعكس، قد يسمح الملف بالقراءة لكن يمنع دليل في مساره الوصول إليه. ويضيف **البت اللاصق** `sticky bit` قيودًا على الحذف في أدلة مشتركة مثل `/tmp`. هذه ليست دعوة لتغيير الأذونات هناك، بل تفسير لعدم كفاية النظر إلى الملف وحده.

> **إضافة تعليمية:** الأذونات ليست حاجزًا كاملًا ضد كل اختراق، لكنها تظل مهمة لتقليل الأثر حتى عند وجود خلل في برنامج. لا نستنتج من احتمال اختراق خدمة أن أذونات الملفات عديمة الفائدة، ولا نستنتج من `rw` أن نظام ملفات مركبًا للقراءة فقط سيسمح بالكتابة.

القناع الافتراضي `umask`

يحجب `umask` أذونات من النمط الذي يطلبه برنامج عند الإنشاء؛ لا يضيف أذونات من العدم، ولا يغيّر الملفات الموجودة. في الحالات العادية التي تطلب فيها أداة ملفًا بنمط `666` أو دليلًا بنمط `777`، يؤدي القناع `022` عادة إلى `644` و`755`، ويؤدي `077` إلى `600` و`700`.

هذا **حجب بتات**، لا طرح عشري يصلح تعميمه على كل حالة، وقد تؤثر أذونات موروثة أو ACL في النتيجة. لتجربة لا تغيّر قناع صدفتك الأم، استخدم صدفة فرعية بالأقواس:

```
(
    umask 077
    touch private-demo
    mkdir private-dir
    ls -ld private-demo private-dir
)
```

يجب أن يكون الاسمان جديدين لكي تلاحظ أثر القناع؛ touch على ملف قديم لا يعيد تعيين أذوناته. وتعود الصدفة الأم إلى قناعها السابق عند انتهاء المجموعة. يصبح القناع إعدادًا لجلسات لاحقة فقط إذا ضُبط في موضع بدء مناسب، وهو موضوع الفصل الثالث عشر.

2.17.2 الروابط الرمزية

الرابط الرمزي اسم يحتوي على مسار إلى اسم آخر، ليس نسخة مستقلة من بياناته. قد يقود إلى ملف أو دليل أو رابط آخر، وقد يكون الهدف غير موجود أصلًا. المثال التالي شكل توضيحي لقائمة طويلة:

```
lrwxrwxrwx 1 amina users 5 Sep 7 10:00 shortcut -> file1
```

الحرف l يعرّف النوع، والسهم يبين نص الهدف. إذا اختفى file1، بقي الرابط نفسه لكن الوصول المعتاد إلى محتواه يفشل؛ يسمى هذا رابطًا معلقًا أو مكسورًا. لا تنفِ وجوده لمجرد أن قراءة هدفه تفشل. افحص الرابط بواسطة ls -l، ويمكن عرض نص هدفه بـreadlink.

إضافة تعليمية: يُفسر هدف نسبي للرابط انطلاقًا من **الدليل الذي يحتوي على الرابط**، لا من دليل عملك وقت استخدامه. وهذه نقطة حاسمة عند إنشاء رابط داخل دليل فرعي. كما أن الأذونات الظاهرة للرابط في لينكس ليست إذنًا لقراءة الهدف بغض النظر عن أذونات ذلك الهدف.

تحرير ملف عبر رابط يعدّل الهدف، فلا تتعامل مع الرابط كنسخة احتياطية. ويمكن أن تتكون سلسلة روابط أو حلقة تؤدي إلى خطأ؛ لا تحاول حلها بحذف أسماء لا تعرف دورها.

2.17.3 إنشاء الروابط الرمزية

تأتي الوجهة أولًا ثم اسم الرابط في الأمر ln -s:

```
ln -s file1 shortcut
ls -l shortcut
readlink shortcut
cat shortcut
```

يفترض المثال أن file1 موجود وأن shortcut اسم جديد داخل دليل التجربة نفسه. ولرابط داخل دليل فرعي، يجب أن يعبر المسار النسبي من مكان الرابط إلى الهدف الصحيح.

افحص ترتيب الوسائط، وخصوصًا إذا كانت الوجهة الثانية دليلًا موجودًا؛ قد ينشئ ln رابطًا داخله بدل إنشاء الاسم الذي تخيلته. وإذا حذفت s- فأنت تطلب **رابطًا صلبًا**، لا رابطًا رمزيًا: اسمًا آخر للكيان نفسه في نظام الملفات ضمن القيود المناسبة، لا ملفًا يحتوي على مسار. سنؤجل تفاصيله إلى فصل أنظمة الملفات.

تستحق الروابط هذا الانتباه لأنها مفيدة في التنظيم والمشاركة وتوفير أسماء مستقرة. والهدف ليس تجنبها، بل معرفة متى تصل إلى البيانات نفسها عبر أكثر من اسم.

2.18 أرشفة الملفات وضغطها

هناك مهمتان مختلفتان: **الأرشفة** تجمع ملفات وأدلة وبيانات وصفية في حاوية واحدة، و**الضغط** يغير تمثيل البيانات لتقليل الحجم غالبًا. يجمع بعض التنسيقات المهمتين، لكن الفصل بينهما يفسر العلاقة بين tar وgzip.

2.18.1 الأداة gzip

يضغط gzip ملفًا، ويستخدم عادة اللاحقة gz.. وبالصيغة التقليدية gzip file يستبدل الملف، عند النجاح المعتاد، بنسخته المضغوطة. يفك gunzip file.gz الضغط ويعيد ملفًا دون تلك اللاحقة. **تحقق من السلوك قبل التجربة، لأن النسخة الأصلية أو المضغوطة قد تزال ضمن العملية.**

لإنشاء نسخة مضغوطة مع الاحتفاظ بالملف التجريبي الأصلي:

```
gzip -c file1 > file1.gz
gunzip -c file1.gz > restored.txt
```

تكتب c- إلى الإخراج القياسي، فتتحكم في الوجهة. وقد تستبدل < ملف وجهة موجودًا؛ لا تستخدم أسماء مهمة. ولا يعني الامتداد وحده أن الملف صالح أو آمن.

2.18.2 الأداة tar

يجمع tar عدة ملفات في أرشيف واحد. لإنشاء أرشيف من الدليل التجريبي dir:

```
tar cvf archive.tar dir
```

تعني c إنشاء، وv عرض الأسماء أثناء العمل، وf أن وسيطًا يحدد اسم الأرشيف. في هذه الصيغة يأتي archive.tar بعد مجموعة الخيارات، ثم dir بوصفه المحتوى المراد جمعه. اللاحقة اتفاق تسمية وليست شرطًا لتفسير الملف.

يمكن حذف v لإخراج أقل، وقد تضيف vv تفاصيل أكثر. يرد في وصف الأصل ذكر r قرب شرح هذا المثال؛ المقصود بشرح الإخراج التفصيلي هنا هو v. لا تبدل الأحرف اعتمادًا على تشابهها: r وضع مختلف للإلحاق في السياقات التي يدعمها tar.

عرض المحتويات قبل الاستخراج

```
tar tvf archive.tar
```

يختار t عرض قائمة المحتويات دون استخراجها. افحص أسماء الأعضاء وبنيتهم: هل يجمعهم دليل علوي متوقع أم سينتشرون في الدليل الحالي؟ قراءة القائمة تكشف مشكلات أولية، لكنها ليست تحققًا من هوية المرسل أو دليلًا على خلو المحتوى من الضرر.

الاستخراج في موضع محدد

لأرشيف أنشأته أنت من بيانات المختبر، جهّز دليلًا فارغًا ثم استخرج إليه:

```
mkdir unpacked
tar xvf archive.tar -C unpacked
```

تعني x استخراجًا، وتحدد C- دليل الوجهة. يبقى الأرشيف بعد الاستخراج. ويمكن اختيار أعضاء محددين بإضافة أسمائهم الدقيقة كما تظهر في القائمة، بدل تخمين أن أسماء الملفات على القرص هي نفسها داخل الأرشيف.

> **إضافة تعليمية أمنية:** لا تستخرج أرشيفًا مجهولًا بصلاحيات المدير أو فوق بيانات مهمة. استخدم دليلًا فارغًا خاصًا لا يستطيع غير الموثوقين تعديل أبويه، وافحص الروابط والمسارات والمحتوى. لا تستخدم خيارات مثل P- للمسارات المطلقة أو خيارات الاستبدال غير المعتادة دون فهم. استخراج الأرشيف ليس أمرًا بتشغيل البرامج الموجودة فيه.

يحاول p- عند الاستخراج المحافظة على أذونات الأرشيف بدل تقييدها بالقناع المعتاد؛ وتختلف بعض افتراضات الأذونات والملكية عند العمل كمدير. لا تضفه تلقائيًا إلى أرشيف غير موثوق، ولا ترفع الصلاحيات لمجرد اختلاف مالك ملف مستخرج. دع الاستخراج يكمل عمله واقرأ رسائله وحالة خروجه؛ قد تؤجل الأداة بعض ضبط أذونات الأدلة حتى يكتمل استخراج محتواها.

إذا كان اسم الأرشيف - بعد خيار الملف، يستخدم tar الإدخال أو الإخراج القياسي بحسب الوضع. سنستفيد من ذلك عند وصل الأرشفة بالضغط.

2.18.3 الأرشيفات المضغوطة .tar.gz

اقرأ الامتداد من الخارج إلى الداخل: gz. طبقة ضغط تحيط بأرشيف tar.. الصيغة التعليمية على مرحلتين هي:

```
gunzip file.tar.gz
tar xvf file.tar
```

يعرض السطران ترتيب الفك، لا وصفة لتشغيلهما على ملف مجهول في دليلك الحالي. بعد فك الضغط، افحص القائمة باستخدام tar tvf وحدد وجهة فارغة للاستخراج. وللإنشاء تسير بالعكس: تجمع الملفات بـtar ثم تضغط الأرشيف بـgzip.

الاسم tgz. يستخدم عادة للتنسيق نفسه بصيغة لاحقة أقصر. لا تحتاج إلى اسم كهذا كي تستخدم الضغط؛ المهم المحتوى وطريقة معالجته.

2.18.4 الأداة zcat

تتيح zcat فك ضغط gzip إلى الإخراج القياسي؛ وفي بيئة GNU تعادل وظيفتها المقصودة هنا gunzip -dc. تعني d- فك الضغط وc- الكتابة إلى الخرج. يمكن وصل الناتج مباشرة إلى tar بدل إنشاء ملف tar. وسيط.

لتجريب الفكرة على بيانات معروفة، أنشئ أولًا أرشيفًا مضغوطًا من دليل المختبر:

```
tar czvf archive.tar.gz dir
```

يختصر خيار z في tar GNU تشغيل الضغط مع وضع الإنشاء c. بعد وجود الأرشيف، يمكن عرض قائمته عبر فك الضغط إلى أنبوب، **دون استخراج**:

```
zcat archive.tar.gz | tar tvf -
```

يقرأ tar البيانات غير المضغوطة من الإدخال القياسي لأن اسم الأرشيف -. وللاستخراج يمكن استخدام x بدل t مع وجهة آمنة. ويوفر الخيار z اختصارًا للفك أيضًا، مثل عرض القائمة مباشرة هكذا:

```
tar tzvf archive.tar.gz
```

تساعدك معرفة الصيغة الطويلة على فهم طبقتي الأرشفة والضغط، ثم يسمح الاختصار بالعمل بأقل كتابة. لا يعني أي من عرضي القائمة إزالة الأرشيف أو استخراج أعضائه إلى الدليل الحالي.

2.18.5 أدوات ضغط أخرى

اللاحقة الشائعة	الضغط	فك الضغط	خيار GNU tar الملائم
.gz	gzip	gunzip	z
.bz2	bzip2	bunzip2	j
.xz	xz	unxz	J

السرعة والحجم واستهلاك الذاكرة تعتمد على البيانات والإعدادات، فلا نجعل المقارنة القديمة بين الأدوات قانونًا ثابتًا. وxz الذي وصفه الأصل بأنه جديد ليس إضافة حديثة عند قراءة هذه الدفعة.

توجد أيضًا zip وunzip للتعامل مع أرشيفات ZIP الشائعة، وهي تجمع عادة الأرشفة والضغط. وقد تستطيع بعض أدوات الفك قراءة أرشيف ZIP ذاتي الاستخراج، لكن **اللاحقة** exe. **لا تعني دائمًا أنه أرشيف، ولا تجعل تشغيل ملف تنفيذي مجهول خطوة لازمة لفحصه.**

أما Z. التاريخية فترتبط بأداة compress. يستطيع gunzip GNU فك بعض هذه الملفات، لكن gzip لا ينشئ ذلك التنسيق. ستقابل تنسيقات أحدث أيضًا، اقرأ دليل الأداة بدل تخمين الخيارات من الامتداد.

2.19 أساسيات بنية الأدلة في لينكس

يمكنك الآن استكشاف الشجرة دون تعديل ملفات النظام. يصف **معيار بنية نظام الملفات** FHS أدوار أدلة شائعة، لكنه لا يضمن أن كل توزيعة أو حاوية أو جهاز مضمن يضع كل ملف في الموضع نفسه. افهم الدور أولًا ثم افحص بيئتك.

خريطة مختصرة؛ المسارات نفسها تبقى من اليسار إلى اليمين

```
/
|-- bin
|-- dev
|-- etc
|-- home
|-- lib
|-- proc
|-- sbin
|-- sys
|-- tmp
|-- usr
|   |-- bin
|   |-- include
|   |-- lib
|   |-- local
|   |-- sbin
|   `-- share
`-- var
    |-- log
    `-- tmp
```

الشكل 2-2 — إعادة رسم تعليمية لفكرة شجرة الأدلة. لا يحصر المخطط جميع الأدلة، ولا يميز وحده بين دليل مستقل ورابط إلى دليل آخر.

من أهم الأدلة الواقعة مباشرة تحت /:

المسار	الدور الذي يساعدك على فهمه
/bin	أوامر أساسية قابلة للتشغيل، مثل أدوات القراءة والنسخ؛ قد يكون رابطًا إلى /usr/bin
/dev	واجهات وملفات أجهزة؛ ليست كلها ملفات بيانات عادية

`/etc`	إعدادات النظام والمضيف؛ وليست مساحة لحفظ مستنداتك
`/home`	أدلة منازل المستخدمين العاديين في ترتيب شائع
`/lib`	مكتبات ومكونات لازمة للبرامج والنظام؛ قد يكون رابطًا ضمن ترتيب حديث
`/proc`	واجهة افتراضية لمعلومات العمليات والنواة؛ كثير من محتواها ليس مخزنًا كملفات قرص عادية
`/sys`	واجهة افتراضية لمعلومات الأجهزة ومكونات النواة وإعدادات مرتبطة بها
`/sbin`	أدوات إدارة النظام في التقسيم التقليدي؛ وجود الأداة هنا لا يمنح منفذها امتيازًا بذاته
`/tmp`	مساحة مؤقتة مشتركة بسياسات وصول وتنظيف؛ لا تحفظ فيها ما تحتاج بقاءه
`/usr`	معظم البرامج والمكتبات والبيانات المشتركة للنظام؛ ليس دليل منزل المستخدم
`/var`	بيانات تتغير أثناء العمل، مثل السجلات والذاكرات المخبئية وبيانات خدمات

قد يحتاج كثير من أدوات الإدارة إلى صلاحيات خاصة، لكن بعضها يتيح معلومات للقراءة بحساب عادي، وقد يتضمن `PATH` لدى ذلك الحساب `/sbin` أو لا يتضمنه. لا نستنتج الصلاحيات من اسم الدليل وحده.

> **توضيح حديث — دمج `/usr`:** تعتمد أنظمة معاصرة، ومنها Debian في سياستها الحالية، روابط من أدلة مثل `/bin` و `/lib` و`/sbin` إلى نظائر تحت `/usr`. لا يعني اختلاف هذه البنية عن الرسم التاريخي أن التثبيت تالف. افحص بـ `ls -ld /bin /sbin /lib`، ولا تحاول تفكيك الروابط لتعيد صورة كتاب قديم.

تسمح `/tmp` عادة بإنشاء ملفات لمستخدمين متعددين، لكن هذا لا يعني حرية قراءة ملفات الجميع أو حذفها. استخدم أسماء آمنة وأدلة خاصة كما فعلنا مع `mktemp`، وتذكر دور البت اللاصق والأذونات. وقد تكون المساحة ذاكرة مؤقتة أو جزءًا من تخزين آخر؛ تجنب ملأها ولا تفترض سياسة تنظيف واحدة.

يوجد `/var/tmp` عادة لملفات مؤقتة أطول عمرًا من ملفات `/tmp`، ولا يشترط مسحه عند كل إقلاع وفق الغرض التقليدي. لكنه ليس نسخًا احتياطيًا ولا وعدًا بالبقاء إلى الأبد؛ قد تنظفه سياسات الإدارة.

2.19.1 أدلة أخرى تحت الجذر

يذكر الأصل أيضًا:

- `/boot`: صور نواة وملفات مرتبطة بالإقلاع ومحمّل الإقلاع. ليس الموضع العام لإعداد كل الخدمات التي تبدأ لاحقًا.

- `/media`: نقطة شائعة لإتاحة وسائط قابلة للإزالة؛ قد تستخدم التوزيعة ترتيباً آخر أو أدلة ضمنها باسم المستخدم.
- `/opt`: موضع شائع لبعض البرمجيات الإضافية أو حزم طرف ثالث، وقد يكون قليل الاستخدام في نظامك.

إضافة تعليمية: ستصادف كذلك `/run` لبيانات تشغيلية مؤقتة منذ الإقلاع، و`/mnt` لنقاط تركيب مؤقتة ينظمها المدير، و`/root` لمنزل الحساب الإداري. ليست هذه الأسماء دعوة للكتابة فيها؛ نستعرضها لتعرف أين تبحث عندما يشير التوثيق إليها.

2.19.2 الدليل `/usr`

ستجد عددًا كبيرًا من البرامج تحت `/usr/bin`، وأدوات الإدارة تحت `/usr/sbin` في التقسيم المعتاد، ومكتبات أو بيانات تنفيذ مساندة تحت `/usr/lib` وفروعه. وقد تستخدم التوزيعة أدلة خاصة بالمعمارية للمكتبات.

ومن المواضع الأخرى:

المسار الكامل	وظيفته المعتادة
`/usr/include`	ترويسات تستخدمها أدوات بناء البرامج، خصوصًا برامج C
`/usr/local`	شجرة لبرامج يثبتها المدير محليًا، منفصلة عن إدارة حزم التوزيعة بحسب السياسة
`/usr/share`	بيانات قابلة للمشاركة وغير مرتبطة عادة بمعمارية تنفيذ بعينها
`/usr/share/man`	صفحات الدليل
`/usr/share/info`	توثيق Info
`/usr/share/doc`	وثائق وأمثلة الحزم

يعرض الأصل أسماء مثل `/include` و`/man` عند وصف فروع `/usr`، وقد تقابل مسارات تاريخية مثل `/usr/man`. **توضيح تحريري:** نكتب المسار الكامل هنا حتى لا يظن القارئ أن `/include` دليل مطلوب مباشرة تحت الجذر. ووجود `/usr/share` لا يعني أنه مركب تلقائيًا عبر الشبكة، أو أن كل ما فيه يعمل بلا اختلاف على جميع أنظمة يونكس.

2.19.3 موضع النواة

قد توجد صورة النواة في مسار مثل `/boot/vmlinuz-...`، وقد يوجد اسم أو رابط مثل `/vmlinuz`. يحمّل مسار الإقلاع الصورة المناسبة ثم يبدأ تنفيذها. تختلف التفاصيل في الأجهزة المضمنة والحاويات وأنظمة الإقلاع المختلفة؛ ليست الحاوية بالضرورة مالكة لصورة نواة داخل شجرتها.

لا يحتاج تنفيذ النواة في الذاكرة إلى إعادة قراءة ملف الصورة في كل تعليمة، لكن **هذا ليس إذنًا بحذف صور الإقلاع.** سيحتاجها النظام عند الإقلاع التالي أو في مسارات إدارة وتحديث أخرى.

توجد وحدات نواة يمكن تحميلها عند الحاجة عادة تحت `/lib/modules` أو الموقع الذي يقود إليه هذا المسار، مع فصل حسب إصدار النواة. يمكن أن يحتوي الجهاز وحدات لأكثر من إصدار. سنناقش الإقلاع والأجهزة لاحقًا؛ يكفي الآن أن تميز ملف صورة النواة عن عملية مستخدم وعن وحدات يمكن تحميلها أثناء العمل.

2.20 تشغيل أوامر بصلاحيات المستخدم الأعلى

يتيح `su` الانتقال إلى هوية أخرى عند سماح المصادقة والسياسة، ويستعمل تقليديًا للدخول إلى جلسة `root`. لكن الجلسة الطويلة ذات الصلاحيات الواسعة تزيد احتمال تنفيذ أمر عادي في سياق خطر، وقد تغير البيئة التي تعمل بها.

ينبه الأصل إلى مشكلات مشاركة كلمة مرور المدير وضعف تتبع مَن غيّر ماذا. **توضيح حديث:** تفاصيل المصادقة والسجلات والبيئة تعتمد على الأدوات والسياسات؛ ليس دقيقًا افتراض أن كل استخدام لـ`su` بلا سجلات، أو أن كل جلسة مدير تسجل تلقائيًا كل ما يحدث داخلها. نفضل منح الصلاحية اللازمة لمهمة محددة، لا رفع الجلسة كلها بلا حاجة.

2.20.1 الأداة `sudo`

تتيح `sudo` تنفيذ أمر بهوية أخرى وفق سياسة مقررة، وتكون الهوية المستهدفة افتراضيًا `root` في الاستخدام المعتاد. إذا لزمت كلمة مرور، فهي عادة كلمة مرور المستخدم الذي استدعى الأمر، لا كلمة مرور المدير، لكن السياسة قد تغير ذلك.

مثال للتعرف، **ليس خطوة لازمة في المختبر ولا أمرًا نجريه آليًا:**

```
sudo id
```

يعرض `id` الهوية التي نفذ بها الأمر إذا سمح له. ويمكن أن يعرض `sudo -l` ما تسمح به السياسة للمستخدم، مع احتمال طلب المصادقة. لا تطلب إضافتك إلى سياسة واسعة من أجل تجربة لم تتطلب إدارة أصلًا.

يستعمل الأصل `sudo vipw` مثالًا لتحرير بيانات الحسابات. نفهم العلاقة بين `sudo` والأداة الإدارية، **ولا نجعل تعديل حسابات النظام تمرينًا هنا.** تختلف وجهة سجلات `sudo` وإعداداتها، فلا نفترض خدمة تسجيل أو facility ثابتة مثل `local2` على كل جهاز. كما أن تتبع الأمر الإداري لا يساوي بالضرورة تتبع كل فعل داخلي في برنامج معقد شغّلته.

2.20.2 الملف `/etc/sudoers`

يحدد ملف sudoers وسياساته المساندة مَن يحق له تنفيذ أي أوامر وعلى أي مضيف وبأي هوية. قد تأتي بعض الإعدادات من ملفات مضمنة أو مصدر مركزي، فلا تعدل ملفًا واحدًا ثم تفترض أنك رأيت السياسة كلها.

المثال الآتي من الأصل للتفسير فقط، وهو واسع الصلاحيات ولا نوصي بنسخه إلى نظام فعلي:

```
User_Alias ADMINS = user1, user2
ADMINS ALL = NOPASSWD: ALL
root ALL=(ALL) ALL
```

يسمي السطر الأول مجموعة أسماء مستعارة ADMINS تضم مستخدمين. في السطر الثاني يحدد ALL الأول المضيفين المطابقين، ويعني ALL بعد وسم المصادقة السماح بكل الأوامر. يلغي NOPASSWD طلب كلمة المرور لهذه القاعدة. وفي السطر الثالث تشير (ALL) إلى إمكان اختيار أي هوية مستخدم مستهدفة بحسب القاعدة.

إضافة (ALL) إلى قاعدة ADMINS توسع الهويات التي يمكن أن تستهدفها كذلك. **المشكلة ليست خطأً في الصياغة؛ بل أن سياسة صحيحة نحويًا قد تمنح سلطة أكثر بكثير من المطلوب.** بعض الأوامر المسموح بها قد يتيح تحرير ملفات أو تشغيل أوامر أخرى، لذلك لا يكفي أن تبدو قائمة الأوامر قصيرة كي تكون محدودة الأثر.

لرؤية بنية قاعدة أضيق نظريًا، لا لتثبيتها تلقائيًا:

```
user1 ALL=(root) /usr/bin/id
```

يفترض هذا المثال صحة مسار البرنامج ويقصر الأمر على أداة فحص الهوية، دون وسم إلغاء المصادقة. إعداد سياسة فعلية يتطلب مراجعة البرامج ووسائطها وملفاتها القابلة للتعديل والسياسات الأخرى، لا استبدال اسم المستخدم في مثال.

استخدم visudo للتحرير الإداري المقصود؛ فهو يقفل الملف أثناء التعديل ويجري فحوصًا أساسية وصياغية. وتوجد صيغة visudo -c للفحص دون تحرير. لا تتجاوز رسالة خطأ بالقوة، ولا تعتمد على فحص الصياغة بوصفه مراجعة أمنية للامتيازات. راجع sudoers(5) وsudo(8) وتوثيق visudo، واترك هذا الملف دون تعديل في المختبر.

2.21 إلى أين نتجه بعد ذلك؟

أصبحت لديك أدوات للدخول إلى فضاء المستخدم عمليًا: تشغيل برامج، والتنقل في الشجرة، وفحص النصوص، وتوصيل تدفقات البيانات، وقراءة الأذونات والعمليات ورسائل الخطأ. تعرف الآن أيضًا أن القدرة على تنفيذ أمر لا تعني أن تنفيذه آمن أو مناسب.

لا يلزم إتقان كل خيار. الأهم أن تستطيع تحديد دليل العمل والوجهة والصلاحية ومصدر البيانات قبل التنفيذ، وأن تفسر الناتج بعده. إذا فهمت لماذا يختلف grep عن توسعة الصدفة، ولماذا قد يفرغ > ملفًا، ولماذا لا يضمن & بقاء عملية بعد الخروج، فقد تجاوزت مرحلة النسخ دون فهم.

يبدأ الفصل الثالث من الأصل بواجهات الأجهزة وملفاتها، مستفيدًا من هذه الأدوات. **الفصل الثالث لم يُعد بعد في هذه الدفعة.** سننتظر مراجعة المستخدم للمقدمة والفصلين الأول والثاني قبل الانتقال إليه.

مختبر ختامي: خمس تجارب مترابطة

> **إضافة تعليمية بالكامل.** استخدم دليل العمل المؤقت الذي أنشأته في بداية الفصل، وحسابًا عاديًا. كل كتلة أدناه تجربة مستقلة تنشئ دليلًا جديدًا باسم واضح؛ إن كان الاسم موجودًا تتوقف بدل استبدال بياناتك. لا تضع تجاربك في /etc أو داخل مشروع مهم. تحتاج الأدوات المعتادة مثل Bash وGNU coreutils وgrep وdiff وtar وgzip وprocps.

تحصر الأقواس ( ) وتغييرات دليل العمل والمتغيرات والقناع في صدفة فرعية. وتعني exit || هنا الخروج من تلك الصدفة إن فشل الأمر السابق، لا مواصلة الكتابة في موضع غير مقصود. لا تستخدم أسلوبًا مشابهًا في ملف آخر دون فهم السياق. ولا تتضمن التجارب حذفًا تكراريًا أو تعديلًا للحسابات أو لـsudoers.

التجربة الأولى: من ملفات صغيرة إلى أنبوب

```
(
    mkdir lab-pipeline || exit
    cd lab-pipeline || exit
    printf '%s\n' beta alpha beta gamma > words.txt
    grep -v gamma words.txt | LC_ALL=C sort > sorted.txt
    cat sorted.txt
)
```

المخرجات المتوقعة مع هذه البيانات:

```
alpha
beta
beta
```

استبعد `grep` السطر الذي يطابق `gamma`، ورتب `sort` الباقي، وكتبت الصدفة الناتج في ملف جديد. بقي الملف الأصلي دون تعديل. **سؤال:** لماذا لم تختف النسخة الثانية من `beta`؟ لأن ترتيب الأسطر ليس حذفًا تلقائيًا للمكرر.

التجربة الثانية: مَن يفسر النجمة؟

```
(
    mkdir lab-quoting || exit
    cd lab-quoting || exit
    touch 'report one.txt' report2.txt .hidden.txt
    printf '%s\n' report*.txt
    printf '%s\n' 'report*.txt'
    LESSON='quoted value'
    export LESSON
    sh -c 'printf "%s\n" "$LESSON"'
)
```

يعرض الاستدعاء الأول لـ`printf` الاسمين المطابقين، كلًا في سطر، ويحافظ الاسم ذو المسافة على وحدته بعد توسعة النمط. يعرض الثاني النمط حرفيًا. ولا يطابق `report*.txt` الملف `.hidden.txt`. وفي النهاية تقرأ صدفة ابنة متغير البيئة المصدّر وتعرض `quoted value`.

سؤال: هل اقتباس أمر `sh -c` المفرد منع الابن من توسيع `LESSON`؟ لا؛ منع الصدفة الأم من توسيعه مبكرًا، ثم فسّر الابن النص ونفذ توسعته داخل الاقتباس المزدوج.

التجربة الثالثة: خاصية الملف ليست خاصية الرابط

```
(
    umask 077
    mkdir lab-permissions || exit
    cd lab-permissions || exit
    printf '%s\n' 'private notes' > notes.txt
    chmod 600 notes.txt
    ln -s notes.txt notes-link
    cat notes-link
    readlink notes-link
    ls -ld . notes.txt notes-link
)
```

يجب أن ترى النص عبر الرابط، وأن يعرض `readlink` المسار `notes.txt`. افحص أذونات الدليل والملف؛ يفترض في بيئة بسيطة دون أذونات موروثة إضافية أن يكونا خاصين. لا تتوقع أن تتطابق أذونات الرابط الظاهرة مع `600` على الهدف. تختلف أسماء المالكين والأوقات والأحجام بين الأجهزة.

سؤال: إذا حررت `notes-link` بمحرر يتبع الرابط، هل تملك نسخة احتياطية مستقلة في `notes.txt`؟ لا، أنت تصل إلى الهدف نفسه.

التجربة الرابعة: أرشيف معروف، ووجهة فارغة

```
(
    mkdir lab-archive || exit
    cd lab-archive || exit
    mkdir bundle unpacked
    printf '%s\n' 'archive sample' > bundle/note.txt
    tar -czf bundle.tar.gz bundle
    tar -tzf bundle.tar.gz
    tar -xzf bundle.tar.gz -C unpacked
    diff -u bundle/note.txt unpacked/bundle/note.txt
)
```

تعرض القائمة أعضاء مثل `bundle/` و`bundle/note.txt`. ويجب ألا يطبع `diff` فروقًا بين الملف المعروف والنسخة المستخرجة. بقي الأرشيف المضغوط وملف المصدر موجودين. ليست تجربة ناجحة على أرشيف أنشأته أنت دليلًا على أمان كل أرشيف من الإنترنت.

التجربة الخامسة: أرسل إشارة إلى عمليتك وحدها

الصق الكتلة معًا؛ فهي تبدأ عملية انتظار قصيرة نسبيًا، وتوقفها وتستأنفها ثم تطلب إنهاءها. لا تستبدل المتغير برقم رأيته على جهاز آخر.

```
(
    sleep 30 &
    demo_pid=$!
    ps -p "$demo_pid" -o pid,ppid,stat,comm
    kill -STOP "$demo_pid"
    kill -CONT "$demo_pid"
    kill -TERM "$demo_pid"
    status=0
    wait "$demo_pid" || status=$?
    printf 'wait status: %s\n' "$status"
)
```

يظهر معرّف يتغير بين التجارب. تكون نتيجة الانتظار غير صفرية عادة لأن العملية انتهت بإشارة، وقد تظهر `143` في Bash عند الإنهاء بـ`SIGTERM`. ليست هذه النتيجة فشلًا غير متوقع في المختبر؛ هي معلومة عن طريقة انتهاء الابن. لم نحتج إلى `SIGKILL` أو إلى صلاحيات المدير.

احتفظ بملفات المختبر إلى أن تراجعها، ثم نظّف فقط الأدلة التي تعرف أنك أنشأتها. لا نقدم أمر تنظيف شاملًا بنمط نجمة كي لا تتحول تجربة آمنة إلى حذف غير مقصود.

أسئلة للمراجعة

1. لماذا لا نفترض أن `/bin/sh` هو Bash على كل جهاز؟
2. ما الفرق بين `cat file1` و`cat < file1` من حيث مَن يفتح الملف؟
3. لماذا تقتبس النمط في `find . -name '*.txt' -print`؟
4. كيف تختلف `*` في الصدفة عن `.*` في تعبير `grep`؟
5. هل يستطيع متغير صدفة غير مصدَّر أن يظهر تلقائيًا لبرنامج خارجي؟ وهل يستطيع الابن تعديل بيئة أمه؟
6. لماذا يمكن أن يكون وضع الدليل الحالي أول `PATH` خطيرًا؟
7. لماذا يختلف `command > out 2>&1` عن `command 2>&1 > out`؟
8. هل يعني وقت المعالج القليل في `ps` أن العملية بدأت منذ وقت قصير؟

9. هل يضمن `&` بقاء العمل بعد إغلاق الطرفية؟ وما الفرق بين Ctrl-Z وCtrl-C؟
10. ماذا تعني أذونات `711` على دليل؟ ولماذا لا يغيّر `umask` أذونات ملف قديم؟
11. لماذا لا يعد الرابط الرمزي نسخة احتياطية، ولا قائمة `tar -t` برهانًا على أمان أرشيف؟
12. هل يكفي نجاح فحص `visudo` للاطمئنان إلى قاعدة `NOPASSWD: ALL`؟

إجابات مقترحة

1. يشير المسار إلى تنفيذ يوفر واجهة الصدفة المطلوبة؛ قد يكون `dash` أو غيره، ولBash إضافات لا يلزم أن يدعمها الجميع. افحص التنفيذ واكتب السكربت للواجهة التي يعلن استخدامها.
2. في صيغة اسم الملف تتعامل `cat` مع فتحه. وفي صيغة `<` تفتحه الصدفة وتوصله بالإدخال القياسي. قد تتشابه النتيجة الظاهرة رغم اختلاف الطريق.
3. لكي يطبق `find` النمط على الأسماء التي يزورها، بدل أن توسعه الصدفة في الدليل الحالي قبل استدعائه. الاقتباس يحفظ النمط حتى يصل إلى الجهة المقصودة.
4. النجمة في توسعة الأسماء تطابق تتابع محارف وفق قواعد المسارات. وفي التعبير النمطي تكرر النجمة العنصر السابق؛ لذلك يجمع `*.` بين «أي محرف» و«تكراره». السياقان مختلفان.
5. لا يمرر المتغير غير المصدَّر كمتغير بيئة بالطريقة المعتادة لتشغيل برنامج خارجي. يحصل الابن على بيئة تخصه، ولا تغير تعديلات نسخته بيئة الصدفة الأم مباشرة.
6. قد يحتوي الدليل على برنامج باسم أداة مألوفة، فيُشغّل بدل النسخة التي توقعتها. تصبح المخاطرة أكبر في دليل يكتب فيه آخرون أو أثناء عمل إداري.
7. تعالج التوجيهات بالترتيب. في الأولى تتبع الأخطاء الوجهة الجديدة للخرج؛ في الثانية تنسخ وجهته القديمة ثم تُغيَّر وجهة الخرج وحده.
8. لا؛ يقيس `TIME` زمن تنفيذ التعليمات على المعالج، لا العمر الزمني للعملية. يمكن لعملية قديمة أن تقضي معظم وقتها في الانتظار.
9. لا يضمن ذلك. قد تؤثر إشارات الخروج وسياسات الجلسة. يعلق Ctrl-Z العمل الأمامي عادة، بينما يرسل Ctrl-C مقاطعة قد تنهيه؛ تعليق العمل ليس الخروج منه.
10. يملك المالك الأذونات الثلاثة، ويملك الآخرون العبور دون سرد الأسماء وفق باقي الشروط. أما `umask` فيقيد إنشاء كيانات جديدة، وليس أمرًا لإعادة تعيين أنماط موجودة.
11. الرابط يقود إلى الهدف نفسه، ففقدان الهدف أو تغييره يؤثر في الوصول عبره. وقائمة الأرشيف تعرض البنية، لكنها لا تثبت مصدره ولا سلامة محتواه وروابطه وامتيازاته.

12. لا؛ الفحص الصياغي لا يحوّل امتيازًا واسعًا إلى سياسة آمنة. يحتاج نطاق الأوامر والهويات والمصادقة والموارد القابلة للتعديل إلى مراجعة مستقلة.

خلاصة عملية قبل إغلاق الطرفية

اسأل عن **المكان والبيانات والصلاحيات**: أين تعمل؟ ما الأسماء التي ستتوسع؟ أي ملف سيفتح للكتابة؟ باسم مَن يجري التنفيذ؟ ثم افحص النتيجة: خرج عادي أم تشخيص؟ نجاح أم حالة خروج لها معنى خاص؟

احتفظ بعادة التجربة على بيانات صغيرة، وقراءة `man`، والتحقق من الأسماء قبل `rm` أو `chmod` أو الاستخراج. المهارة ليست كتابة أمر قصير بأكبر سرعة، بل معرفة ما سيحدث ولماذا حدث بالفعل.

راجع المعجم وملاحظات الإعداد والمراجع عند الحاجة، ودوّن ملاحظاتك على المقدمة والفصلين الأول والثاني لتُراجع معًا في الدفعة التالية من العمل.

معجم الدفعة الثانية

إضافة تعليمية من الإعداد العربي. هذه تعريفات تمهيدية مرتبطة بالفصلين الأول والثاني، وليست تعريفات معيارية تستوعب كل الحالات. نلتزم بهذه المقابلات في الدفعات اللاحقة، ونوثّق أي تغيير اصطلاحي بدل تبديل الكلمات بصمت.

النظام والتنفيذ

المصطلح الإنجليزي	المقابل العربي	المعنى في هذا الكتاب
Abstraction	التجريد	التركيز على الوظيفة والعلاقات مع إرجاء تفاصيل لا يحتاج إليها السؤال الحالي.
Component	مكوّن	جزء من النظام يمكن وصف وظيفته وواجهاته مع الأجزاء الأخرى.
Layer	طبقة	مجموعة مكونات عند مستوى مناسب من التجريد؛ ليست كل طبقة حدًا أمنيًا.
Hardware	العتاد	الأجزاء المادية للحاسوب وأجهزته.
Operating system	نظام التشغيل	المنظومة التي تدير الموارد وتقدم بيئة وخدمات لتشغيل البرامج؛ نحدد نطاق المقصود بحسب السياق.
Distribution	توزيعة	تجميع للنواة والأدوات والمكتبات والخدمات، مع وسائل لتثبيتها وإدارتها.
Kernel	النواة	المكوّن المركزي الذي ينظم موارد النظام ويقدم واجهات محمية للعمليات.
CPU	وحدة المعالجة المركزية؛ المعالج	العتاد الذي ينفذ التعليمات.
CPU core	نواة معالجة	وحدة تنفيذ داخل المعالج؛ ليست نواة نظام التشغيل.
Logical CPU	معالج منطقي	مورد تنفيذ تراه النواة وتستطيع إسناد مهمة إليه؛ لا يساوي بالضرورة نواة مادية مستقلة.
Program	برنامج	تعليمات وبيانات معدّة للتنفيذ، وغالبًا تكون محفوظة في ملف.
Process	عملية	مثيل جارٍ من تنفيذ برنامج، له حالة وموارد ومعرّف.

Process ID / PID	معرّف العملية	رقم يميز العملية ضمن فضاء معرّفات العمليات الذي نلاحظها فيه.
Parent process	العملية الأم	العملية المرتبطة بعلاقة الأبوة بعملية ابنة؛ نستخدم نموذج الإنشاء المعتاد في هذا الفصل.
Child process	العملية الابنة	العملية الناتجة عن الإنشاء في نموذج الأم والابنة.
Thread	خيط تنفيذ	مسار تنفيذ داخل عملية؛ تشترك خيوط العملية في مساحة عناوينها.
Scheduling	الجدولة	اختيار المهام التي تعمل وتوقيت إتاحة المعالج لها وفق سياسة النظام.
Context switch	تبديل السياق	حفظ ما يلزم من سياق تنفيذ واستعادة سياق آخر لمتابعة مهمة مختلفة.
Time slice	شريحة زمنية	حصة زمنية في النموذج المبسط لتقاسم المعالج؛ ليست مدة ثابتة موحدة لكل المهام.
Concurrency	التزامن	تقدم أعمال متعددة خلال فترة واحدة، ولو بالتناوب.
Parallelism	التوازي	تنفيذ أعمال متعددة في الوقت نفسه على موارد تنفيذ متعددة.
Interrupt	مقاطعة	حدث يستدعي معالجة خاصة، مثل حدث مؤقت أو إشارة من جهاز.

الذاكرة والواجهات والصلاحيات

المصطلح الإنجليزي	المقابل العربي	المعنى في هذا الكتاب
Main memory / RAM	الذاكرة الرئيسية	الذاكرة التي تحتوي على التعليمات والبيانات اللازمة للعمل الجاري؛ وهي متطايرة عادة.
Persistent storage	التخزين الدائم	وسيط يحتفظ بالبيانات دون الاعتماد على بقاء الجهاز قيد التشغيل، مثل قرص التخزين.
Bit	بت	وحدة ثنائية تأخذ القيمة صفر أو واحد.

State	حالة	وصف مكوّن أو نظام عند لحظة معينة، على مستوى تفصيل نحدده.
Process image	صورة العملية	تمثيل البرنامج وبياناته في الذاكرة، ليست صورة رسومية.
Address space	فضاء العناوين	مجموعة العناوين التي يمكن تفسيرها في سياق عملية أو بيئة معينة.
Virtual memory	الذاكرة الافتراضية	نموذج يفصل العناوين التي يستخدمها البرنامج عن ترتيب الذاكرة المادي ويتيح تنظيمها وحمايتها.
MMU	وحدة إدارة الذاكرة	عتاد يساعد على ترجمة العناوين وتطبيق حماية الذاكرة.
Page table	جدول الصفحات	بنية تديرها النواة لوصف خرائط صفحات الذاكرة وخصائص الوصول إليها.
Swap	مساحة المبادلة؛ المبادلة	تخزين مساند يمكن استخدامه لبعض صفحات الذاكرة، ليس هو الذاكرة الافتراضية كلها.
Copy-on-Write	النسخ عند الكتابة	مشاركة مادية مؤقتة لصفحات، مع إنشاء نسخة مستقلة عندما تتطلب الكتابة ذلك.
User mode	نمط المستخدم	نمط تنفيذ محدود الامتياز لتعليمات برامج المستخدم.
Kernel mode	نمط النواة	نمط تنفيذ يتيح تعليمات وعمليات ذات امتياز لإدارة النظام.
User space / userland	فضاء المستخدم	بيئة عمليات المستخدم ومجالات ذاكرتها، بما فيها كثير من الخدمات الخلفية.
Kernel space	فضاء النواة	مجال الذاكرة المحمي الخاص بالنواة.
Device driver	برنامج تشغيل جهاز	برمجيات تعرف تفاصيل التعامل مع جهاز وتدمجه في واجهات النظام.
System call / syscall	نداء نظام	واجهة تطلب عبرها عملية خدمة من النواة.
Pseudo-device	جهاز زائف	واجهة برمجية شبيهة بواجهة جهاز، دون اشتراط وجود عتاد مستقل يقابلها.

User / UID	مستخدم / معرّف المستخدم	هوية ومرجع رقمي يستعملهما النظام في بيانات الاعتماد وقرارات الصلاحيات.
Superuser / root	المستخدم ذو الصلاحيات العليا	الحساب الإداري التقليدي ذو المعرّف صفر؛ ليس نواة النظام.
Group / GID	مجموعة / معرّف المجموعة	هوية جماعية تساعد على تنظيم أذونات الوصول والمشاركة.
Permissions	أذونات الوصول؛ الصلاحيات	قواعد تحدد العمليات المسموح بها على مورد أو ضمن سياق معين.
Capabilities	القدرات	وحدات امتياز تفصل بعض الصلاحيات الإدارية إلى عناصر يمكن ضبطها.
Namespace	فضاء أسماء	آلية تقدم للعمليات رؤية معزولة لمورد من موارد النظام، مثل معرّفات العمليات.
Shell	صدفة الأوامر	برنامج يفسر الأوامر ويوفر وسائل لتركيبها وتشغيلها.
Terminal	طرفية	واجهة لإدخال النص وعرضه؛ يمكن أن تعمل الصدفة داخلها.
Directory	دليل؛ مجلد	بنية تربط أسماء بملفات وأدلة أخرى ضمن شجرة نظام الملفات.
Log	سجل	رسائل أو بيانات تسجل أحداثًا للمراقبة والتشخيص.

فروق تستحق بطاقة تذكير

- **العملية ليست البرنامج:** يمكن أن ينتج تشغيل البرنامج نفسه عدة عمليات.
- **النواة ليست نواة المعالج:** الأولى برمجيات إدارة، والثانية عتاد تنفيذ.
- **فضاء المستخدم ليس حساب المستخدم:** الأول بيئة تنفيذ، والثاني هوية وصلاحيات.
- **المستخدم `root` ليس الدليل `/`:** الأول حساب، والثاني جذر شجرة الملفات.
- **الذاكرة الافتراضية ليست المبادلة:** يمكن أن توجد الأولى من دون تفعيل الثانية.
- **نداء النظام ليس أمر الصدفة:** قد يستخدم أمر واحد نداءات نظام كثيرة أثناء تنفيذه.
- **`fork` ليس `exec`:** الأول ينشئ عملية، والثاني يستبدل برنامج عملية موجودة.

إضافات الفصل الثاني: الأوامر ومسارات الملفات

المصطلح الإنجليزي	المقابل العربي	المعنى في هذه الدفعة
Argument	وسيط	قيمة أو اسم يمرر إلى أمر، الخيارات وأسماء الملفات من أمثلته.
Option	خيار	وسيط يغير طريقة عمل أداة، مثل `-l` أو `--help`.
Builtin	أمر مدمج	أمر تنفذه الصدفة نفسها، مثل `cd`، دون اشتراط برنامج خارجي مستقل.
Working directory	دليل العمل الحالي	نقطة الانطلاق التي تفسر منها العملية مساراتها النسبية.
Absolute path	مسار مطلق	مسار يبدأ بـ `/` ويحدد موقعًا انطلاقًا من الجذر.
Relative path	مسار نسبي	مسار يفسر نسبة إلى موضع مرجعي، دليل العمل عادة، أو دليل الرابط عند حل هدف رابط نسبي.
Home directory	الدليل المنزلي	دليل خاص بحساب، وليس مرادفًا للدليل الجذر أو لشجرة `/usr`.
Dotfile	ملف نقطي	اسم يبدأ بنقطة، إخفاؤه من عرض افتراضي لا يوفر حماية أمنية.
Symbolic link	رابط رمزي	كيان يحتوي على مسار إلى اسم آخر، لا نسخة مستقلة من بيانات الهدف.
Hard link	رابط صلب	اسم إضافي للكيان نفسه في نظام الملفات، وليس ملفًا يحتوي على مسار هدف.
Dangling link	رابط معلق، رابط مكسور	رابط رمزي لا يمكن الوصول إلى هدفه المقصود.
Timestamp	طابع زمني	قيمة زمنية مثل وقت التعديل أو الوصول، لا تعني جميع الطوابع الشيء نفسه.
Manual page	صفحة دليل	توثيق مرجعي محلي عادة، ينتمي إلى قسم مثل `1` أو `5`.
FHS	معيار بنية نظام الملفات	معيار يصف أدوار أدلة شائعة، لا يلغي اختلاف التوزيعات وطرق تنظيمها.
Merged /usr	دمج شجرة `/usr`	تنظيم تستخدم فيه مسارات تقليدية مثل `/bin` روابط إلى نظائرها تحت `/usr`.

إضافات الفصل الثاني: اللغة وتدفقات البيانات

المصطلح الإنجليزي	المقابل العربي	المعنى في هذه الدفعة
Expansion	توسعة	استبدال تركيب من لغة الصدفة بقيم أو أسماء قبل تنفيذ الأمر.
Globbing	توسعة أنماط أسماء الملفات	مطابقة نمط مثل `*.txt` مع الأسماء، وليس البحث عن سطور داخل الملفات.
Regular expression	تعبير نمطي	لغة مطابقة للنصوص، تختلف قواعدها عن توسعة الأسماء في الصدفة.
Quoting	اقتباس	ضبط كيفية تفسير الصدفة للنص، وحماية المحارف أو تجميع القيمة في وسيط واحد.
Shell variable	متغير صدفة	قيمة تحتفظ بها الصدفة؛ لا تُمرر بالضرورة إلى برنامج خارجي.
Environment variable	متغير بيئة	قيمة مصدّرة يمكن تمريرها إلى برنامج ابن؛ نسخته لا تعدل بيئة الأم مباشرة.
PATH	مسار البحث عن الأوامر	قائمة أدلة مرتبة للبحث عن برامج خارجية عند عدم تحديد مسار صريح.
Stream	تدفق	تسلسل بيانات يقرأه برنامج أو يكتبه عبر واجهة إدخال/إخراج.
Standard input / stdin	الإدخال القياسي	قناة الإدخال ذات الواصف `0` في الاتفاق المعتاد.
Standard output / stdout	الإخراج القياسي	قناة النتائج العادية ذات الواصف `1`.
Standard error / stderr	الإخراج القياسي للأخطاء	قناة التشخيص ذات الواصف `2`؛ ليست جزءًا من الخرج العادي تلقائيًا.
File descriptor	واصف ملف	رقم يعرّف مورد إدخال/إخراج مفتوحًا داخل سياق العملية.
Redirection	إعادة توجيه	تغيير ارتباط قناة بملف أو بواصف آخر، مثل `>` أو `2>&1`.
Pipe	أنبوب	وصل خرج عملية بإدخال أخرى دون اشتراط ملف دائم وسيط.
Clobber / truncate	استبدال المحتوى؛ تفريغ الملف	إزالة محتوى وجهة موجودة قبل الكتابة الجديدة في سياق مثل `>`.
Exit status	حالة الخروج	قيمة تصف نهاية أمر؛ تفسيرها مرتبط باتفاق الأداة، لا بكونها غير صفرية فقط.

إضافات الفصل الثاني: المهام والأذونات والأرشفة

المصطلح الإنجليزي	المقابل العربي	المعنى في هذه الدفعة
Signal	إشارة	إشعار لعملية أو مجموعة عمليات، قد يطلب الإنهاء أو التوقف أو الاستئناف.
Job control	التحكم في المهام	إدارة الصدفة لمهامها الأمامية والخلفية والمتوقفة، رقم المهمة ليس PID.
Foreground job	مهمة أمامية	مهمة تشغل المقدمة في جلسة تفاعلية وتتفاعل عادة مع الطرفية.
Background job	مهمة خلفية	مهمة تتيح للصدفة متابعة استقبال الأوامر، لا يضمن ذلك استمرارها بعد الخروج.
File mode	نمط الملف	معلومات نوع الملف وبتات الأذونات والبتات الخاصة المرتبطة بها.
Octal mode	نمط ثماني	تمثيل عددي للبتات، مثل `600` أو `755`، كل خانة تجمع قيم القراءة والكتابة والتنفيذ.
umask	قناع إنشاء الملفات	بتات تحجب من الأذونات المطلوبة عند الإنشاء، ولا تعدل الكيانات القديمة.
setuid	بت تعيين هوية المستخدم	بت خاص قد يؤثر في هوية التنفيذ الفعالة عند تشغيل برنامج، وفق القيود والسياسات.
Sticky bit	البت اللاصق	قيد خاص على إزالة الأسماء وإعادة تسميتها في أدلة مشتركة مثل `/tmp`.
Archive	أرشيف	حاوية تجمع ملفات وأدلة وبيانات وصفية، لا يشترط أن تكون مضغوطة.
Compression	ضغط	تمثيل للبيانات يهدف عادة إلى تقليل الحجم، ليس مرادفًا لجمع عدة ملفات.
Extraction	استخراج	إعادة إنشاء أعضاء أرشيف في وجهة محددة، مع الانتباه للمسارات والأذونات.
sudoers policy	سياسة `sudoers`	قواعد تسمح لمستخدم بتنفيذ أوامر بهوية أخرى ضمن حدود يحددها المدير.
Passphrase	عبارة مرور	سر أطول قد يتكون من عدة كلمات، قوة الاختيار والعشوائية وعدم إعادة الاستخدام أهم من المظهر وحده.

ملاحظات الإعداد والمراجع

المصدر وحدود التغطية

المصدر هو **How Linux Works: What Every Superuser Should Know**، **الطبعة الثانية**، تأليف **Brian Ward**، نشر **No Starch Press**. نعتمد ملف `HowLinuxWorks.pdf` المرفق بالمستودع، وعدد صفحاته **338 صفحة**. تنص صفحة الحقوق، وهي الصفحة 337 من الملف، على حقوق الطبعة الثانية لعام 2015، كما توجد في الملف بيانات نشر من عام 2014.

لا تُشتق إحالات الصفحات هنا من العلامات المرجعية المضمنة في PDF؛ بعض هذه العلامات لا يطابق بداية النص بدقة. جرى الاعتماد على مواضع العناوين في الصفحات نفسها. يعرض ملف `manifest.json` في مشروع الكتاب نطاقات المصدر وحالة كل فصل، بما فيها الفصول غير المعدّة بعد.

مادة هذه الدفعة	موضعها في ملف المصدر	طريقة التعامل معها
المقدمة العربية	المقدمة، الصفحات من 4 إلى 6	إعادة صياغة لأهداف التعلم والجمهور وطريقة القراءة والتنظيم، مع إرشادات عربية إضافية.
الفصل الأول	الصفحات من 8 إلى 14	إعداد تعليمي لجميع أقسام الفصل من 1.1 إلى 1.6، مع الحفاظ على ترتيبها.
الفصل الثاني	الصفحات من 15 إلى 41	جميع الأقسام من 2.1 إلى 2.21 وأقسامها الفرعية؛ 63 عنوانًا مرقّمًا، مع أمثلة وصيغ وملاحظات أمان.
الرسوم الخمسة	الأشكال 1-1 و1-2 و1-3 و2-1 و2-2	مخططات تعليمية جديدة، ليست استنساخًا للتصميم أو لجميع أمثلة الرسوم الأصلية.
جداول الفصل الثاني	الجداول من 2-1 إلى 2-4	المحارف، ومفاتيح التحرير، وأقسام الدليل، وأنماط الأذونات؛ أُعيدت بالعربية مع توضيحات للسياق.
المختبر والأسئلة والإجابات والمعجم	إضافات إلى الفصل	محتوى تعليمي مضاف، وليس ترجمة لتمارين أو معجم في الأصل.
التوضيحات التقنية	مراجع أدناه	تفصل التبسيط التمهيدي عن تفاصيل مهمة في تطبيقات لينكس.

لم تتضمن الدفعة ترجمة عبارات الثناء، أو صفحة شكر المؤلف، أو الفصول من الثالث إلى السابع عشر، أو قائمة مراجع الأصل وفهرسه ونبذة مؤلفه. ذكر أسماء تلك المواد في خريطة المشروع **لا يعني إعداد محتواها.**

مراجع التوضيحات المضافة

المراجع الآتية صفحات توثيق خارجية يمكن الرجوع إليها للتثبت أو التوسع. الرقم بين القوسين في اسم صفحة دليل يحدد قسمها. استُخدمت هذه المراجع للتحقق من التوضيحات ذات الصلة أثناء إعداد هذه المسودة بتاريخ 7 سبتمبر 2026؛ لا يعني ذلك مراجعة بشرية متخصصة للدفعة كلها.

1. صفحة الدليل fork(2): إنشاء العملية الابنة، واستقلال فضاءي العناوين، والنسخ عند الكتابة، والتمييز بين واجهة المكتبة والتنفيذ الداخلي. ترتبط بالقسم 1.3.4.
2. صفحة الدليل execve(2): استبدال صورة البرنامج داخل العملية، والسمات التي تبقى أو تتغير. ترتبط بالقسم 1.3.4.
3. صفحة الدليل clone(2): واجهات إنشاء المهام والتحكم في الموارد المشتركة، ومنها `clone3()`. تبين حدود تعميم نموذج `fork` وحده.
4. صفحة الدليل pthreads(7): تعدد خيوط التنفيذ داخل العملية والموارد التي تتشاركها. ترتبط بتوضيح الجدولة في القسم 1.3.1.
5. صفحة الدليل capabilities(7): تجزئة الامتيازات الإدارية إلى قدرات. ترتبط بالقسم 1.5.
6. صفحة الدليل namespaces(7): الرؤية المعزولة للموارد وصلتها بالحاويات. ترتبط بالمقدمة والقسم 1.5 وحدود ملاحظة العمليات في المختبر.
7. صفحة الدليل free(1): معنى أعمدة الذاكرة، ولا سيما `available`، وخيار العرض المقروء. ترتبط بتجربة الذاكرة.
8. صفحة الدليل systemd-journald.service(8): وظيفة خدمة جمع السجلات ضمن منظومة `systemd`. ترتبط بالمثال المضاف في القسم 1.4.

تصف صفحات الدليل حالات تفصيلية أكثر من الفصلين المتاحين. لا يلزم إتقانها كلها الآن، ولا ينبغي تحويل النموذج المبسط هنا إلى ضمان مطلق بشأن كل معمارية أو توزيعة أو إعداد أمني.

مراجع التوضيحات في الفصل الثاني

استُخدمت الصفحات الآتية أثناء إعداد الفصل الثاني بتاريخ 7 سبتمبر 2026. لا تعني الإحالة أن الأداة أو سياسة التوزيعة متطابقة على كل جهاز. وتبقى مراجعة المستخدم للمقدمة والفصلين معًا معلقة حتى تصل ملاحظاته؛ الإذن ببدء الفصل الثاني ليس اعتمادًا نهائيًا لما سبقه.

1. سياسة Debian: الملفات والسكربتات: القسمان 10.1 و10.4 يوضحان ترتيب دمج `/usr` ومتطلبات واجهة `/bin/sh`. ترتبط بالأقسام 2.1 و2.19.
2. دليل Bash: توسعة أسماء الملفات: مطابقة الأسماء النقطية وحالات الأنماط غير المطابقة وخيارات `nullglob` و `failglob` و`globskipdots`. يرتبط بالقسمين 2.4.4 و2.7.

3. دليل Bash: إعادة التوجيه: ترتيب التوجيهات، وتفريغ الوجهة، ونسخ واصفات الملفات، وحدود `noclobber`. يرتبط بالقسم 2.14.
4. دليل Bash: الإشارات: `SIGINT` و`SIGHUP` وعلاقة الخروج بالمهام الخلفية؛ يصحح تعميم أن `&` وحدها تضمن الاستمرار. يرتبط بالقسم 2.16.
5. دليل GNU Grep: أنواع المطابقة: الفروق بين المطابقة الأساسية والموسعة والثابتة وخيارات `-E` و`-F`. يرتبط بالقسم 2.5.1.
6. دليل GNU Coreutils: بنية بتات النمط: معاني أذونات الملفات والأدلة، `setuid`، والبت اللاصق، وقيود أخرى على الوصول. يرتبط بالقسم 2.17.
7. دليل GNU Coreutils: البتات الخاصة للأدلة: حدود تعميم أن النمط العددي القصير يمحو كل البتات الخاصة. يرتبط بالقسم 2.17.1.
8. دليل GNU Tar: قواعد أمان عملية: وجهة استخراج خاصة وفارغة، وحماية المسارات الأب، والحذر من الخيارات غير المعتادة. يرتبط بالقسم 2.18.
9. دليل Sudo: سياسة sudoers: المصادقة والتسجيل والهوية المستهدفة وقواعد السياسة. لا نعمم مثال `NOPASSWD: ALL` في الأصل بوصفه إعدادًا مناسبًا للمبتدئ. يرتبط بالقسم 2.20.
10. دليل Sudo: الأداة visudo: القفل والفحص الصياغي وحدودهما، دون الادعاء بأن صياغة صحيحة تعني سياسة آمنة.
11. NIST SP 800-63B: كلمات المرور: كلمات المرور والعبارات الطويلة وقواعد التحقق، بدل الاعتماد على وصفة الاختصار وإضافة الرموز في الأصل. يرتبط بالقسم 2.6؛ لا تعني الإحالة أن حساب لينكس المحلي يطبق سياسة NIST تلقائيًا.

سجل التغطية وحدود الأمثلة

يسجل ملف التغطية `coverage/chapter-02.json` العناوين الإنجليزية والعربية وأرقام الصفحات لكل عنوان مرقم في الفصل الثاني، والأشكال والجداول والمختبرات. هذا سجل مقابلة، وليس بديلًا عن نص الفصل. بعض الأمثلة أعيد تنظيمه على ملفات صغيرة داخل دليل مؤقت بدل افتراض وجود قاموس أو استخدام ملفات النظام للتعديل.

حافظ الإعداد على الفرق بين أنواع الأمثلة: أوامر قراءة، وكتابة على بيانات تجريبية، ومخرجات توضيحية، وصيغ عامة، وإجراءات تفاعلية أو إدارية **لا تُنفذ آليًا**. خمس تجارب ختامية جديدة تُفحص على موارد تنشئها بنفسها، ولا تُشغّل جميع كتل الأوامر لمجرد ورودها في النص. لا يُعد نجاحها اختبارًا لسياسة `sudoers` أو تغيير كلمات مرور أو بقاء خدمة بعد تسجيل الخروج.

تبقى ملفات الدفعة الأولى محفوظة كما سُلّمت. الدفعة الثانية تراكمية، وتحدّث فقط عبارات إتاحة المحتوى في المقدمة والفصل الأول، إلى جانب إضافة الفصل الثاني وتوسيع المعجم والمراجع. لم تُدّعَ مراجعة نهائية للمادة السابقة قبل وصول الملاحظات المشتركة.

سياسة الإعداد والمراجعة

- نكتب بالفصحى، ونشرح المصطلح عند ظهوره الأول، ونثبت مقابله في المعجم.
- نحافظ على أسماء الأوامر والمسارات والخيارات والمعرّفات. لا نترجم محتوى يُفترض أن يفسّره برنامج.
- نميز الأوامر عن المخرجات، ونصرّح بأن المخرجات التوضيحية قد تختلف عن جهاز القارئ.
- نحافظ على أرقام أقسام الأصل. الإضافات التعليمية لا تحصل على أرقام توحي بأنها أقسام أصلية جديدة.
- نوسم التحديث والتصحيح والتوسع بدل نسبته إلى المؤلف أو تحديث الطبعة بصمت.
- لا نضع نصًا إنجليزيًا غير مُعدّ داخل فصل ثم نعلن اكتماله. المواد غير المنجزة تبقى ظاهرة بوصفها «لم يبدأ إعدادها».
- الاختبارات الآلية تتحقق من بنية الملفات والروابط وبناء المخرجات وبعض الأمثلة؛ وهي لا تحل محل مراجعة عربية وتقنية بشرية.

للمراجعة، اذكر اسم الملف وعنوان الفقرة، ثم اقترح التعديل وسببه. فرّق بين خطأ تقني، وخيار اصطلاحي، وتحسين أسلوبي. الهدف أن يتمكن القارئ من فهم الفكرة وتجربتها بأمان، لا أن تبدو العبارة قريبة من ترتيب الكلمات الإنجليزية فقط.

خريطة إنجاز الكتاب

عدد الفصول ذات المسودات: 2 من أصل 17، إضافة إلى المقدمة العربية. هذه أعداد فصول، وليست نسبة مئوية من كلمات الكتاب أو صفحاته. «مسودة متاحة» تعني وجود نص قابل للقراءة والمراجعة، لا اعتماد ترجمة نهائية.

الفصل	العنوان العربي المقترح	صفحات ملف المصدر	الحالة
1	الصورة العامة	8–14	مسودة متاحة
2	الأوامر الأساسية وبنية الأدلة	15–41	مسودة متاحة
3	الأجهزة	42–58	لم يبدأ إعدادها
4	الأقراص وأنظمة الملفات	59–80	لم يبدأ إعدادها
5	كيف تُقلع نواة لينكس	81–94	لم يبدأ إعدادها
6	كيف يبدأ فضاء المستخدم	95–124	لم يبدأ إعدادها
7	إعداد النظام: السجلات والوقت والمهام الدفعية والمستخدمون	125–142	لم يبدأ إعدادها
8	نظرة أعمق إلى العمليات واستخدام الموارد	143–158	لم يبدأ إعدادها
9	فهم الشبكة وإعدادها	159–191	لم يبدأ إعدادها
10	تطبيقات الشبكة وخدماتها	192–209	لم يبدأ إعدادها
11	مقدمة إلى سكربتات الصدفة	210–228	لم يبدأ إعدادها
12	نقل الملفات عبر الشبكة	229–241	لم يبدأ إعدادها
13	بيئات المستخدمين	241–249	لم يبدأ إعدادها
14	جولة موجزة في سطح مكتب لينكس	250–259	لم يبدأ إعدادها
15	أدوات التطوير	260–275	لم يبدأ إعدادها

16	مقدمة إلى بناء البرامج من شيفرة C المصدرية	276–287	لم يبدأ إعدادها
17	البناء على الأساسيات	288–291	لم يبدأ إعدادها

تبدأ بعض الفصول في الصفحة نفسها التي ينتهي فيها الفصل السابق؛ مثل الفصلين 12 و13 في الصفحة 241. لذلك قد تتداخل حدود الصفحات دون تداخل المحتوى.

مواد أخرى لم تُعد بعد: عبارات الثناء، شكر المؤلف، قائمة مراجع الأصل، فهرس الأصل، نبذة عن المؤلف، صفحة الحقوق الأصلية.

الخطوة التالية بعد المراجعة: انتظار ملاحظات المستخدم على الدفعة الحالية، ثم إعداد الفصل 3: الأجهزة. لا يحتوي هذا الإصدار على صفحات فارغة أو ملخصات بديلة تدّعي أنها الفصول المتبقية.